

WHEN SPARSITY BECOMES USEFUL COMPUTATION: PRESSURE, THRESHOLDING, AND SITE PLACEMENT

Anonymous authors

Paper under double-blind review

ABSTRACT

Activation zeros, zero-operand computation, and inference latency can rank sparsification recipes differently. We study these disagreements in 54 Pythia pretraining conditions on MiniPile, varying activation pressure, thresholding, and site placement with matched initialization and data order within each size. Model-wide sparsity measures the fraction of counted scalar multiplications with a zero activation operand; selected-site reach accounts for architectural and inference-workload differences. Pressure’s marginal benefit depends on both threshold and target set. At the largest tested threshold, seven-site thresholding with orthogonal L1 (OL1) pressure reaches 27.48%, 40.60%, and 80.62% model-wide sparsity at 14M, 70M, and 410M, with dense-relative validation-loss penalties of 0.621, 1.116, and 0.573, respectively. These are 91.8%, 82.2%, and 92.4% of selected-site reach. At that 14M endpoint, expanding to seven sites reduces feed-forward (FFN) up-input sparsity while increasing attention sparsity. A specialized kernel profitably exploits projection zero fragments, but attention skipping slows every tested checkpoint. Seven-site’s larger native-relative speedup also coexists with higher absolute optimized latency because its native reference is slower. These single-seed comparisons motivate evaluating quality, computational reach, and execution structure together.

1 INTRODUCTION

Activation sparsity can reduce language-model inference cost when kernels exploit zero operands (Liu et al., 2024; Cetin et al., 2026). But a researcher choosing an intervention under a quality and execution budget faces three different questions: how many activations become zero, how much computation consumes those zeros, and how much time an implementation actually saves. These quantities need not favor the same recipe.

Existing methods combine different mechanisms: TEAL thresholds activations after training, ProSparse applies activation pressure during continued training, and Q-Sparse sparsifies attention and FFN projection inputs (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). Their different models, training protocols and execution settings make mechanism attribution difficult. We study *pressure*, *thresholding*, and *site placement* under a common pretraining protocol. Our contribution is a controlled comparison connecting their conditional effects to activation distributions, zero-operand products, and measured latency, rather than a claim that combining more interventions uniformly improves efficiency.

We pretrain Pythia-family models from scratch for controlled comparisons across sizes (Biderman et al., 2023). Thirty 14M conditions provide local and multisite ablations; twelve conditions at each of 70M and 410M evaluate selected complete recipes independently. We define model-wide sparsity $\mathcal{S}_{\text{model}}$ as the fraction of counted scalar matrix multiplications with a zero activation operand, and selected-site reach $\mathcal{S}_{\text{model}}^{\text{max}}$ as the fraction reachable if the selected sites were entirely zero. The latter depends on architecture and the inference workload, and need not bound natural zeros outside the selected sites.

The comparisons reveal three disagreements. First, pressure’s value changes with its surrounding recipe. At the largest tested threshold, adding OL1 to the seven-site recipe increases $\mathcal{S}_{\text{model}}$ from 15.39% to 27.48% for +0.1265 loss; the complete recipe costs +0.6208 relative to the dense refer-

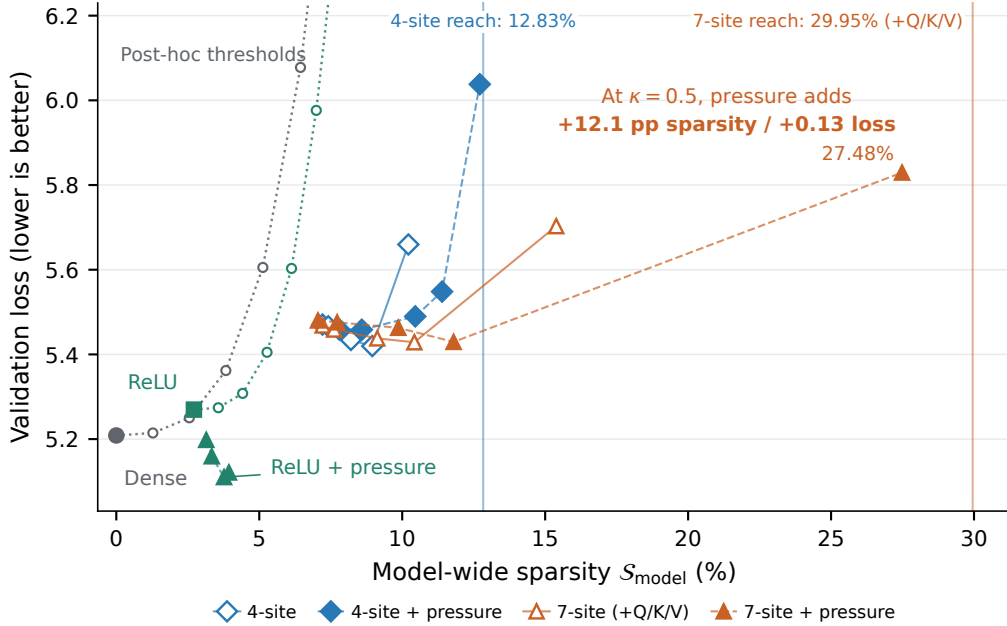


Figure 1: **Quality-sparsity trade-offs at 14M.** Markers show 26 trained conditions (naive L1 omitted). Four-site thresholds act at projection inputs; seven-site thresholds also act at Q/K/V. For these two families, solid/open and dashed/filled encode no pressure and OL1, respectively. Dotted paths apply post-hoc magnitude thresholding at four sites to the dense and one-site ReLU references. Vertical guides mark selected-site reach; the seven-site endpoint reaches 91.8% of its 29.95% reference. Lines connect evaluated settings. Full post-hoc paths appear in Appendix D.3.

ence (Figure 1). Second, seven-site OL1 has fewer zeros at the FFN-up input but greater model-wide sparsity than four-site OL1. Third, that greater model-wide sparsity does not yield the lowest optimized latency in the tested implementation. These disagreements expose distinct choices about quality, computational coverage, and exploitable zero structure.

The $\kappa = 0.5$ quality-sparsity ordering recurs across sizes, but the full threshold response does not. These single-seed comparisons establish conditional recipe relationships, not a quality scaling law. A human-guided coding-agent loop supports kernel implementation (Ouyang et al., 2025; Luo et al., 2026); matched ablations separate projection benefit, attention overhead and native-reference cost.

2 RELATED WORK

Sparsification interventions. Post-hoc magnitude thresholding zeros small activations without updating weights: CATS targets FFNs and TEAL includes attention projections (Lee et al., 2024; Liu et al., 2024). ReLU Strikes Back studies nonlinearity choices (Mirzadeh et al., 2023); ProSparse combines pressure and thresholding during continued training (Song et al., 2024), while Cetin et al. (2026) study sparse pretraining and execution. Q-Sparse and Spark sparsify attention and FFN projection inputs with top- k or statistical selection (Wang et al., 2024; You et al., 2025). We use fixed thresholds to study placement and pressure under matched pretraining, additionally targeting internal Q/K/V operands. This compares mechanisms rather than ranking those complete methods under their original protocols.

Sparse execution. TEAL, TwELL and Spark couple sparsity with implementations adapted to particular shapes and inference settings (Liu et al., 2024; Cetin et al., 2026; You et al., 2025). Memory movement, selection costs and instruction granularity limit whether zeros save time. Coding agents can assist kernel development with execution feedback (Ouyang et al., 2025; Luo et al., 2026). Our qualified timings and matched ablations evaluate learned representations, without claiming agent superiority or state-of-the-art inference performance.

3 METHODOLOGY

3.1 SPARSIFICATION INTERVENTIONS

An intervention specifies thresholding sites $\mathcal{N}$, functions G , and pressure sites $\mathcal{P}$; these sets need not coincide. We use $G_{+, \kappa}(x) = x \mathbf{1}\{x \geq \kappa\}$ and $G_{\pm, \kappa}(x) = x \mathbf{1}\{|x| \geq \kappa\}$, with fixed $\kappa \geq 0$ within a condition. Unlike top- k selection, these rules do not require ranking activation magnitudes. Appendix A specifies boundary and derivative conventions.

Figure 2 locates the sites: a, m are the normalized attention and FFN branch inputs; h is the FFN hidden activation; z is the attention context before output projection; q, k, v are internal attention operands, with q, k after RoPE. The probabilities are p . We use one-sided thresholds at a, m, h, z and symmetric thresholds at q, k, v ; at h , the nonlinearity replaces GELU. *One-, four-, and seven-site* refer to site types repeated in every transformer block, not numbers of layers: $\{h\}$, $\{a, m, h, z\}$, and $\{a, m, h, z, q, k, v\}$.

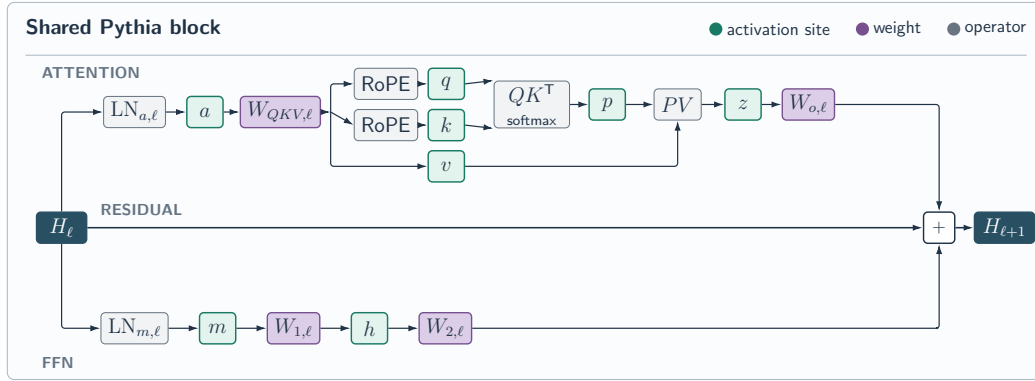


Figure 2: **Site types in each Pythia block.** Attention and FFN branches have separate LayerNorms. Green nodes denote activations and purple nodes weights; Table 1 gives intervention assignments.

The pressure objective $\mathcal{L}_1$ equally averages mean absolute activations across targeted site-layer tensors after thresholding. *Naive L1* minimizes $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$, adding a weight that affects quality and sparsity (Song et al., 2024; Cetin et al., 2026). We propose *orthogonal L1* (OL1), inspired by gradient-conflict projection (Yu et al., 2020; Hsieh et al., 2024): it removes the pressure component opposing the AdamW-preconditioned task direction, then caps the correction norm relative to that direction. Ignoring stabilizers, the capped correction maximizes alignment with the pressure direction within this norm budget and non-opposition constraint. Once the cap binds, increasing λ leaves the ideal correction unchanged at a fixed optimizer state. We fix multisite $\lambda = b = 1$ for tractability and measure cap activity; saturation is not universal. This construction does not guarantee task-loss preservation (Appendix A.2).

3.2 SPARSITY AND SELECTED-SITE COMPUTATIONAL REACH

Activation sparsity is the exact-zero fraction at a site. *Model-wide sparsity* is $\mathcal{S}_{\text{model}} = N_0/N_{\text{model}}$, where N_0 counts scalar multiplications with a zero activation operand in block matrix multiplications. The denominator includes those products and the dense final vocabulary projection at every token. Counts pool layers and validation sequences, excluding future-masked attention pairs. Softmax, normalization, data movement and other non-matrix operations are excluded; $\mathcal{S}_{\text{model}}$ is a logical opportunity, not saved time (Appendix B).

The *selected-site reach ceiling* $\mathcal{S}_{\text{model}}^{\text{max}} = N_{\text{reach}}(\mathcal{N})/N_{\text{model}}$ measures coverage when all selected sites are entirely zero. It depends on architecture, sequence length and output scope. Natural zeros outside this reach can make $\mathcal{S}_{\text{model}} > \mathcal{S}_{\text{model}}^{\text{max}}$, so $\mathcal{S}_{\text{model}}/\mathcal{S}_{\text{model}}^{\text{max}}$ is a reference ratio, not necessarily a bounded utilization. Table 1 reports reach for the declared full-sequence inference setting; Appendix B.2 defines the counters.

4 EXPERIMENTAL STUDY

We pretrain randomly initialized Pythia-14M, 70M, and 410M models (Biderman et al., 2023). To fit our compute budget, we adapt Pythia’s training recipe to MiniPile (Kaddour, 2023), using 1.493 billion input tokens per model. We conduct extensive comparisons at 14M and promote the sparsest recipes to 70M and 410M, alongside their baselines. We measure validation cross-entropy loss and $\mathcal{S}_{\text{model}}$ over all complete sequences from MiniPile’s validation documents. Table 1 maps the eight recipes onto the site types in Figure 2.

Table 1: **Sparsification recipes and selected-site reach.** Reach values are $100 \cdot \mathcal{S}_{\text{model}}^{\max} \%$ at $T = 2,048$. A0 is the dense GELU reference; A1-H is one-site ReLU at h ; A4/A7 use the thresholding functions in Section 3.1. A1-H-L1 uses L1N; the OL1 suffix denotes orthogonal pressure. All recipes are evaluated at 14M; 70M and 410M use A0, A1-H, A4-OL1 and A7-OL1.

ID	Thresholding sites $\mathcal{N}$	Pressure sites $\mathcal{P}$	Reach (%)		
			14M	70M	410M
A0	$\emptyset$	$\emptyset$	0.00	0.00	0.00
A1-H	h	$\emptyset$	4.28	12.35	24.93
A1-H-L1	h	h	4.28	12.35	24.93
A1-H-OL1	h	h	4.28	12.35	24.93
A4	a, m, h, z	$\emptyset$	12.83	37.06	74.78
A4-OL1	a, m, h, z	a, m, h, z	12.83	37.06	74.78
A7	a, m, h, z, q, k, v	$\emptyset$	29.95	49.42	87.25
A7-OL1	a, m, h, z, q, k, v	a, m, h, z, q, k, v	29.95	49.42	87.25

The experimental setup contains 30 training runs at 14M and 12 at each larger model size. Within each size, runs share initialization, data order and training budget. At 14M, we vary local pressure for A1-H using L1N and OL1 with $\lambda \in \{0.05, 0.1, 0.5, 1\}$. For A4/A7, we sweep $\kappa \in \{0, 0.01, 0.05, 0.1, 0.5\}$, both without pressure and with OL1 at $\lambda = b = 1$. Figure 4 characterizes the resulting cap activity at 14M. There is one training seed per condition, and all quality comparisons reuse the same validation split. Conditions are not independent-seed replications. The larger cohorts contain only the pressure-on multisite sweeps; they do not isolate the effect of adding pressure. Appendix C.4 details the training settings.

For post-hoc magnitude thresholding, we zero activations below uniform percentile targets $p = 0, 0.1, \dots, 0.9$ at a, m, h, z as a TEAL-style baseline (Liu et al., 2024). Site-layer thresholds are calibrated separately on ten training blocks.

4.1 QUALITY-SPARSITY TRADE-OFFS

Local pressure and broader thresholds offer different trade-offs (Figure 1). One-site ReLU + OL1 at $\lambda = 0.5$ reaches 3.768% sparsity and loss 5.1102, versus dense loss 5.2086. Post-hoc magnitude thresholding of the dense reference reaches similar sparsity, 3.832%, at loss 5.3619. Broader trained recipes reach larger $\mathcal{S}_{\text{model}}$, but seven-site placement also has more computational reach than the four-site post-hoc comparator: 29.952% versus 12.833% at 14M. Thus the broad comparison changes both learned representation and reachable computation; it does not establish universal dominance over matched-site post-hoc methods. One-site OL1 is also not uniformly better than naive L1: their small loss differences reverse at $\lambda = 1$ (Appendix Table 6).

4.2 PAIRED EFFECT OF ADDING PRESSURE

We compare pressure-on and pressure-free recipes at each matched κ (Figure 3). Four-site OL1 slightly improves loss at $\kappa = 0$ and 0.01, adding 0.598 and 1.173 pp of $\mathcal{S}_{\text{model}}$. Its increment reaches 2.498 pp for +0.3783 loss at $\kappa = 0.5$. Seven-site OL1 adds 1.372 pp for +0.0008 loss at $\kappa = 0.1$, but 12.096 pp for +0.1265 at 0.5. **Pressure’s marginal benefit depends on both threshold and target set.**

Define the complete-recipe interaction $I_S(\kappa) = (S_{7,+} - S_{7,-}) - (S_{4,+} - S_{4,-})$, where + and − denote OL1 and no pressure; define I_L analogously for loss. At $\kappa = 0.5$, $I_S = +9.598$ pp

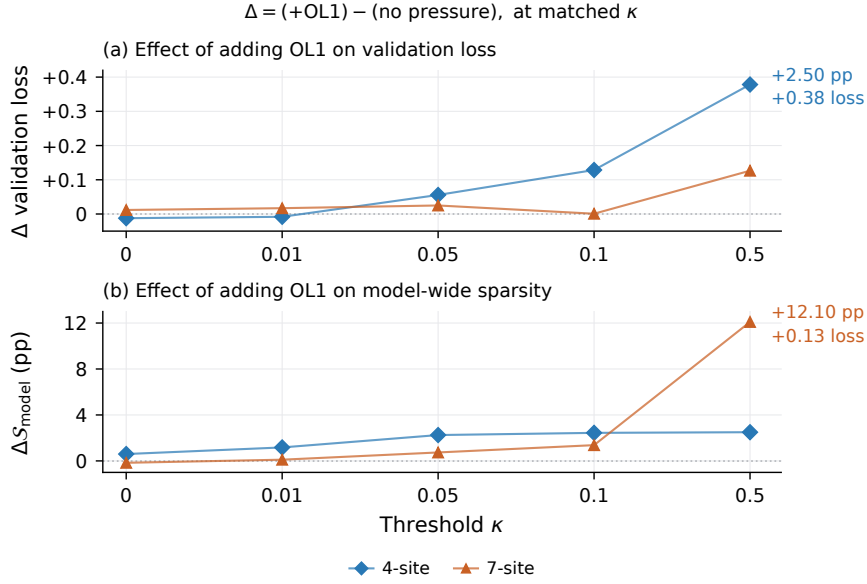


Figure 3: **Conditional effects of adding OL1.** Each point subtracts the pressure-free endpoint from its OL1 counterpart at the same trained κ : A4-OL1 minus A4, or A7-OL1 minus A7. Panels show validation-loss change and model-wide sparsity change (pp). At $\kappa = 0.5$, the four-site effect is +2.50 pp/+0.38 loss and the seven-site effect is +12.10 pp/+0.13 loss. These are marginal pressure costs, not total penalties relative to the dense model. Lines order separately trained settings, with matched initialization, data order and budget.

and $I_L = -0.2518$. Expanding to seven sites changes both pressure targets and their equal-tensor normalization, so this is not an isolated effect of Q/K/V pressure. The large difference occurs at the largest sampled threshold; the gap from 0.1 to 0.5 does not identify a transition or its location.

The same pressure weight also produces different optimization regimes (Figure 4). For adaptive task and pressure directions u, w , the task-relative opposing component is $\rho_{\text{opp}} = \max(0, -\langle u, w \rangle) / (\|u\|_2^2 + \epsilon)$. Across 7,120 logged steps, 99.9% have negative alignment. Median ρ_{opp} is 0.0095 for four-site and 0.588 for seven-site pressure; at $\lambda = b = 1$, the cap is active on 0.5% and 97.0% of steps. This is a larger task-relative opposing component in OL1’s AdamW-preconditioned coordinates, not a direct measure of loss sensitivity or the fraction of pressure removed. The geometry combines direction and magnitude and does not identify a cause of the distribution changes below.

4.3 BROADER THRESHOLD PLACEMENT WITHOUT PRESSURE

Pressure-free four-/seven-site comparisons keep κ matched while adding Q/K/V thresholds (Table 2). At zero threshold these added nonlinearities are identities, and the endpoints nearly coincide. At $\kappa = 0.5$, broader thresholding adds 5.17 pp for +0.043 loss. This control separates threshold placement from the complete-recipe pressure interaction.

Table 2: **Four- versus seven-site thresholds without pressure.** A4 targets a, m, h, z ; A7 additionally targets q, k, v . Values are raw matched endpoints, not pressure effects.

κ	A4 loss	A7 loss	A4 S_{model} (%)	A7 S_{model} (%)
0	5.4705	5.4684	7.212	7.218
0.01	5.4665	5.4588	7.414	7.618
0.05	5.4341	5.4379	8.206	9.127
0.1	5.4197	5.4287	8.953	10.425
0.5	5.6597	5.7029	10.216	15.387

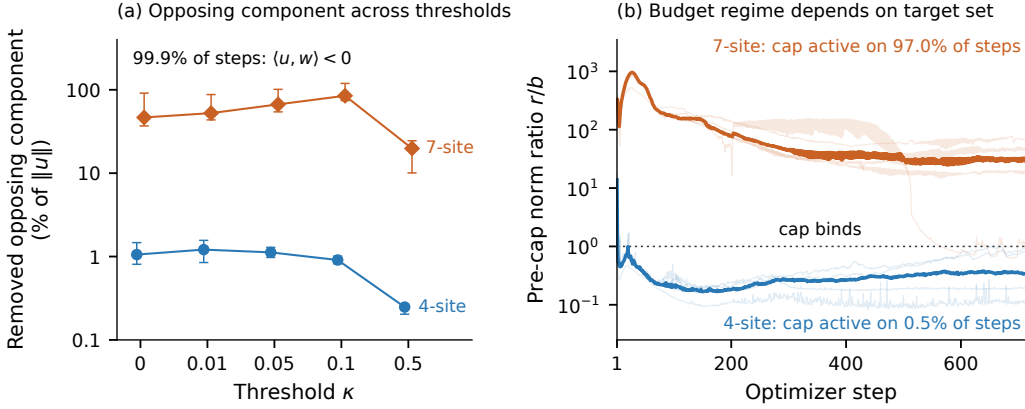


Figure 4: **Pressure targets change OL1 geometry and cap activity.** (a) Median $100\rho_{\text{opp}}$ over 712 steps per condition; whiskers are the 25th–75th percentiles. (b) Pre-cap norm ratio at $\lambda = b = 1$; faint curves are five thresholds per target set, thick curves their per-step median. Cap-active fractions use the implemented scale $s < 1$. Geometry is measured before group learning rates and decoupled weight decay; it does not guarantee task-loss preservation (Appendix A.2).

4.4 LOCAL SPARSITY DOES NOT DETERMINE MODEL-WIDE SPARSITY

At $\kappa = 0.5$, expanding four-site OL1 to seven sites reduces exact zeros at the **FFN-up input m from 96.77% to 68.70%**, whereas the FFN hidden activation h changes only from 99.94% to 99.88% (Figure 5). These are measured per-site counts, not estimates from rounded operation contributions. Pooling h, m would weight them 80:20 and obscure the distinction. Layer-level counts appear in Appendix Figure 10.

Despite fewer zeros at m , seven-site OL1 reaches 27.48% model-wide sparsity versus 12.71% for four-site, with lower loss (5.8294 versus 6.0380). The decomposition explains why: $QK + PV$ contribute 16.77 pp versus 0.06 pp, outweighing smaller projection contributions. FFN-up contribution falls from 4.139 to 2.939 pp, while FFN-down remains approximately 4.27 pp. The pressure-free bars show the corresponding placement comparison without OL1.

Four-site OL1’s $Q/K/V$ density concentrates near zero, yet only 0.22% of these activations are exactly zero, versus 95.60% with seven-site OL1. Since four-site recipes do not directly target $Q/K/V$, their difference from the dense reference is a *nonlocal response of the trained network*. It does not isolate pressure, thresholding, or their interaction. Likewise, the FFN-up change may reflect target reweighting, cap behavior, or representation adaptation; the present controls do not distinguish them.

4.5 CROSS-SIZE RECIPE EVALUATION

At the largest tested threshold, $\kappa = 0.5$, seven-site OL1 has lower loss and greater S_{model} than four-site OL1 at all three sizes (Figure 6). At $\kappa = 0$, four-site is better on both axes at 14M and 70M, but seven-site is better at 410M. This is recurrence of one complete-recipe ordering, not transfer of the full threshold response. The larger cohorts lack pressure-free multisite controls; exact contrasts are in Appendix Table 7.

Seven-site reach rises from 29.95% to 49.42% to 87.25%, partly because the dense vocabulary head contributes a smaller share as width and depth grow. The $\kappa = 0.5$ endpoints reach 27.48%, 40.60%, and 80.62% S_{model} : 91.8%, 82.2%, and 92.4% of these references. Their dense-relative loss penalties are +0.6208, +1.1162, and +0.5732 (perplexity factors 1.86, 3.05, and 1.77). Thus the larger raw sparsity at 70M does not imply an improved quality cost or reference ratio. The seven-site coverage formula in Appendix C.2 makes its architecture-and-workload dependence explicit.

Dense losses themselves are non-monotonic: 5.2086, 4.0998, and 4.5475. The fixed token budget supplies 106.14, 21.20, and 3.68 tokens per parameter; 410M also has a lower peak learning rate.

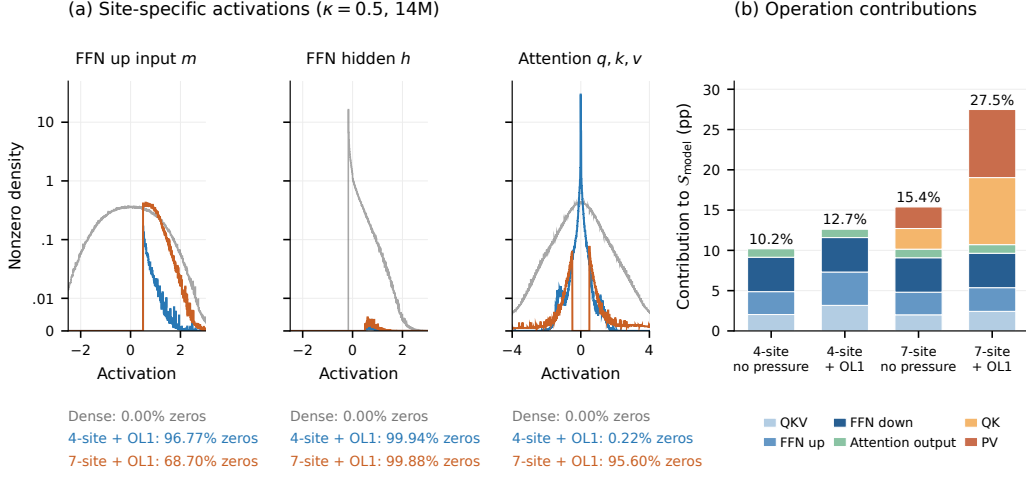


Figure 5: **Site-specific zeros and operation-weighted consequences.** (a) Nonzero densities at m , h , and pooled Q/K/V, separately, for the dense reference and four-/seven-site OL1 at $\kappa = 0.5$; exact-zero percentages appear below. Counts pool validation sequences and layers; density divides by all elements and bin width, without renormalizing the nonzero mass. (b) Four-/seven-site endpoints with and without OL1 at $\kappa = 0.5$. Stacks use the common full-model denominator, including the dense vocabulary head. Bars measure zero-product opportunities, not time savings. Pressure-free density measurements are unavailable; their bars use the retained logical counters.

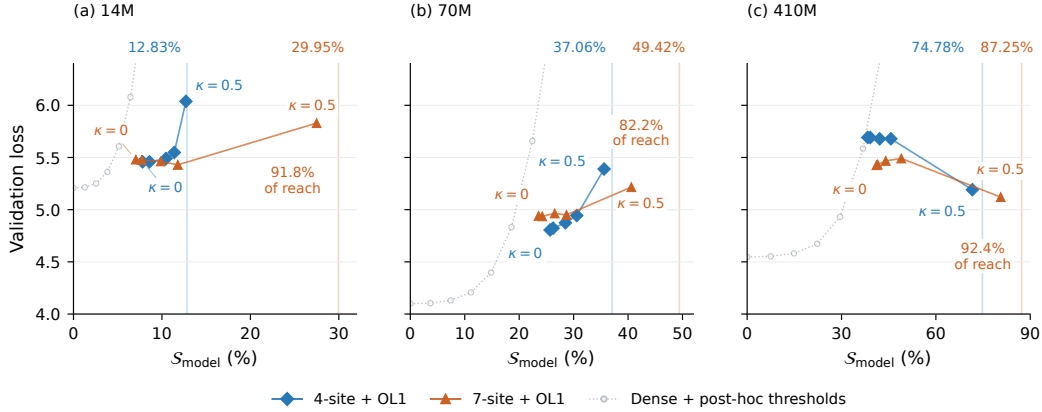


Figure 6: **The $\kappa = 0.5$ complete-recipe ordering recurs across sizes.** Absolute loss versus S_{model} for independently trained four-/seven-site OL1 conditions; points use $\kappa \in \{0, 0.01, 0.05, 0.1, 0.5\}$. Gray dotted curves apply four-site post-hoc magnitude thresholding to the dense references. Vertical guides mark selected-site reach for full-sequence, full-vocabulary inference; endpoint annotations are reference ratios. Axes share the loss range but use size-specific sparsity ranges. Lines connect evaluated settings; complete post-hoc paths are in Appendix D.3.

Its different realized optimization regime is not evidence of premature convergence or failure to converge (Appendix Figure 8). These experiments test within-size recipe relationships rather than a quality scaling law.

4.6 WHEN MODEL-WIDE SPARSITY TRANSLATES TO SPEEDUP

We evaluate all 30 Pythia-14M checkpoints with the same specialized K050 implementation on one RTX5090: BF16, batch-size-one, uncached 2,048-token inference with full-vocabulary logits. Complete-validation numerical qualification precedes timing on 64 fixed inputs, seven paired passes

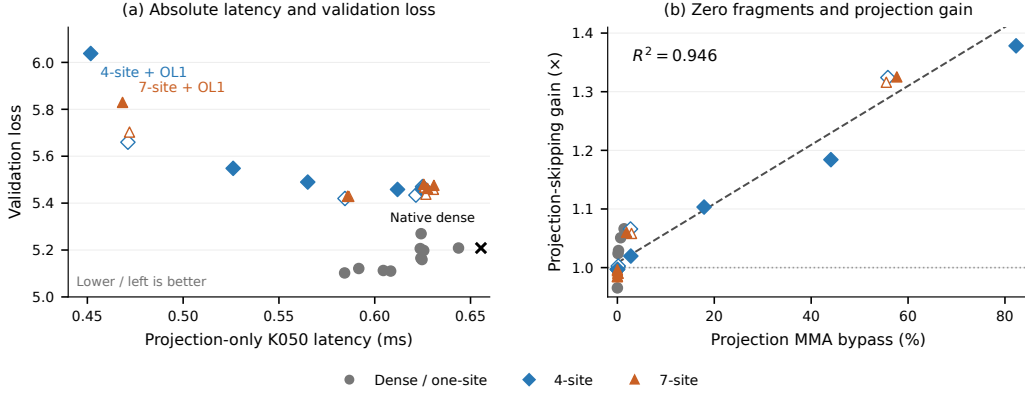


Figure 7: **Quality, absolute latency and exploitable zero structure.** (a) All 30 checkpoints under projection-only K050; the black cross marks native dense-reference latency. Labels identify the $\kappa = 0.5$ OL1 pair. Loss is the canonical validation measurement; BF16 execution passes the qualification bounds. (b) Projection MMA bypass versus matched gain t_0/t_P ; the dashed line is descriptive OLS. For four-/seven-site points, open means no pressure and filled means OL1. Gray points include the dense reference, one-site ReLU, naive L1 and OL1; their fill does not encode pressure. Bypass includes scalar substitution and padded instruction work. No equal-quality interpolation or causal fit is assumed.

and three processes (Appendix D.4). We use absolute latency versus validation loss as the deployment comparison, and recommend the *projection-only* variant for this measured cohort: disabling attention skipping improves every checkpoint.

The sparsest model is not the fastest optimized model (Figure 7a). At $\kappa = 0.5$, four-site OL1 takes 0.45175 ms with loss 6.0380; seven-site OL1 takes 0.46837 ms with loss 5.8294. Neither dominates the other on quality and latency. With both sparse paths enabled, their latencies are 0.45948 and 0.47337 ms. Yet seven-site reports a larger native-relative speedup, $1.783\times$ versus $1.609\times$, because its native reference is slower: 0.84409 versus 0.73944 ms. The native implementation executes additional Q/K/V gates separately, whereas K050 fuses them into existing operations. This supports an overhead explanation without isolating each operation’s cost.

Let t_N, t_0, t_P, t_K be native, optimized all-skips-off, projection-only, and full K050 latency. The ratio decomposes as

$$\frac{t_N}{t_K} = \frac{t_N}{t_0} \frac{t_0}{t_P} \frac{t_P}{t_K}; \quad 1.783 \simeq 1.360 \times 1.325 \times 0.989$$

for seven-site OL1 at $\kappa = 0.5$. The first factor includes dense-kernel and layout changes as well as fusion. Projection skipping is profitable; attention skipping is not. A common native A0 reference instead gives $1.427\times$ and $1.385\times$ for the four-/seven-site full K050 pair; these cross-model ratios still include implementation changes and quality differences (Appendix D.5).

Instruction bypass tracks projection benefit more closely than scalar zeros alone. Across the cohort, projection MMA bypass is associated with projection-skipping gain at $R^2 = 0.946$, compared with 0.413 for the projection scalar zero-product fraction. Entire activation fragments can permit bypassing matrix multiply-accumulate (MMA) instructions; very sparse rows may instead use scalar arithmetic. These are kernel-exploitable zero-fragment patterns, not an imposed structured-sparsity constraint. At $\kappa = 0.5$, four-site OL1 bypasses 82.30% of projection MMAs versus seven-site’s 57.68%.

Seven-site’s additional attention opportunity supplies the boundary case. The same checkpoint skips 58.00% of eligible QK and 67.21% of PV instructions, but attention skipping adds 5.00 microseconds overall. Detection and control recur after fragment loads, and zero QK scores still participate in softmax. Counts of skipped instructions therefore need not translate into net time savings. This is a

property of the tested kernel, shapes and hardware, not evidence that attention sparsity is intrinsically unprofitable.

The implementation was developed through a human-guided coding-agent loop configured with GPT-6 Astra (OpenAI, n.d.); search history and compatibility checks are in Appendix D.4. The scientific result is conditional realization: projections expose profitable skipping, while additional attention sparsity and a larger native-relative ratio do not identify the lowest-latency representation.

5 DISCUSSION AND CONCLUSION

Selecting an intervention requires local zero fractions, operation accounting, validation quality and absolute execution time. Their rankings can disagree: pressure’s value depends on threshold and target set, and broader attention sparsity need not yield lower latency. Native-relative speedup also confounds recipe comparison with each reference’s cost.

These conclusions are descriptive. The 54 configurations use one training seed per condition and the same validation split, not 54 independent replications or an untouched confirmation evaluation. Seven-site OL1 changes pressure targets and objective normalization together; its norm-cap activity does not identify why individual activation distributions change. The $\kappa = 0.5$ result is the largest sampled threshold, with no observations between 0.1 and 0.5. Cross-size comparisons hold tokens, not tokens per parameter, fixed, use a lower 410M learning rate, and lack larger-model pressure-free multisite controls. They establish recurrence of one within-size ordering, not improving quality cost with scale.

The kernel results concern one RTX5090 and batch-size-one, uncached full-sequence inference. Projection skipping is profitable at selected settings; attention skipping is not, even where instructions are bypassed. Stronger compiled dense baselines, cached decoding, other shapes and devices, independent seeds, and independent confirmation data remain untested. The study consequently supports conditional intervention choices and identifies where further controls or implementation work are needed.

AI USE STATEMENT

Generative AI tools assisted methodology discussion, mathematical exposition, implementation, retained-data analysis and interpretation, figure generation, literature lookup, manuscript drafting and revision. Kernel development used a human-guided coding-agent loop configured with GPT-6 Astra; the recorded configuration does not identify the model served for every request. Research choices and experiment launches were directed by the human researcher. Numerical kernel qualification, source-provenance checks and focused code tests supported verification of AI-assisted artifacts. The authors are responsible for the final text, claims and artifacts.

REPRODUCIBILITY STATEMENT

Appendices A–C specify the interventions, counters, data preparation and realized training protocol. Appendix D.4 gives numerical qualification and timing; Appendix D.6 describes the supplied measurements and source hashes. The accompanying data support reanalysis but are not a standalone executable checkpoint and kernel release.

REFERENCES

- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 2397–2430. PMLR, 2023. URL <https://proceedings.mlr.press/v202/biderman23a.html>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 19085–19100. PMLR, 2024. URL <https://proceedings.mlr.press/v235/hsieh24a.html>.
- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=v3w2a7EInO>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.

- OpenAI. GPT-6 Astra. OpenAI API documentation, n.d. URL <https://developers.openai.com/api/docs/models/gpt-6-astra>. Accessed September 10, 2026.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 47356–47415. PMLR, 2025. URL <https://proceedings.mlr.press/v267/ouyang25a.html>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

A INTERVENTION DEFINITIONS

A.1 THRESHOLDING NONLINEARITIES

For finite $\kappa \geq 0$, both thresholding nonlinearities retain equality and leave retained values unchanged. The comparison mask is detached: the derivative with respect to the input is one for retained entries and zero for rejected entries. Consequently, $G_{+,0}$ and ReLU have identical forward values but differ at zero, where their derivatives are one and zero, respectively. $G_{\pm,0}$ is exactly the identity in both values and gradients. These nonlinearities use neither a straight-through estimator nor a top- k selection rule. Unselected sites retain their original transformations.

A.2 ACTIVATION PRESSURE AND OL1

For the J targeted site-layer tensors A_j , containing n_j entries each, the pressure objective is $\mathcal{L}_1 = J^{-1} \sum_{j=1}^J n_j^{-1} \sum_{i=1}^{n_j} |A_{j,i}|$. The tensors contain the executed site activations, after the non-linearity where one is applied. Absolute values and means are evaluated in float32. Each tensor receives equal weight regardless of its size; adding pressure sites changes both the objective and the weight of existing targets. Task and pressure losses are averaged over the same accumulation microbatches. For naive L1, the accumulated gradient of $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$ is globally norm-clipped before AdamW (Loshchilov & Hutter, 2019).

For OL1, let g_t and g_1 be the accumulated task and unweighted pressure gradients. We first clip g_t by its global norm and perform AdamW using only that clipped task gradient. Let $\hat{m}_t$ and $\hat{v}_t$ be its newly updated, bias-corrected moments. On parameters having both gradients and initialized optimizer state, define the adaptive task and pressure vectors $u = \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon_A)$ and $w = g_1 / (\sqrt{\hat{v}_t} + \epsilon_A)$. The pressure gradient is unclipped and has no first-moment buffer. Inner products and norms below pool all these eligible parameters. With $c = \langle u, w \rangle$, $q = \|u\|_2^2$, and numerical stabilizer $\epsilon > 0$, the conflict-conditioned projection is

$$\tilde{w} = \begin{cases} w - \frac{c}{q + \epsilon} u, & c < 0 \text{ and } q > \epsilon, \\ w, & \text{otherwise.} \end{cases} \quad (1)$$

For diagnostics, we define the task-relative opposing component

$$\rho_{\text{opp}} = \frac{\max(0, -c)}{q + \epsilon}.$$

When projection is active, this equals the norm of the component removed from w , divided by $\|u\|_2$; it is zero when $c \geq 0$. All reported steps satisfy the guard $q > \epsilon$. It is therefore not the fraction of the pressure norm removed.

For pressure weight $\lambda \geq 0$ and required budget $b > 0$, define the norm ratio $r = \lambda \|\tilde{w}\|_2 / (\|u\|_2 + \epsilon)$. Set $s = \min\{1, b/(r + \epsilon)\}$ when $r > 0$, or $s = 1$ otherwise. After AdamW, each eligible parameter group p , with learning rate α_p , receives $\theta_p \leftarrow \theta_p - \alpha_p \lambda s \tilde{w}_p$. The experiments' clipping thresholds, b , and λ are condition parameters. We report the norm budget as active when the implemented scale satisfies $s < 1$.

Ignoring numerical stabilizers, the correction norm before the learning rate is $\min\{\lambda \|\tilde{w}\|_2, b\|u\|_2\}$. For a fixed optimizer state, once the cap binds, further increases in λ leave this ideal correction unchanged.

This procedure is related to gradient-conflict projection (Yu et al., 2020) and the use of smoothed primary directions (Hsieh et al., 2024), but defines a particular correction in AdamW-preconditioned coordinates. The stabilized projection is only approximately orthogonal under conflict. Its geometry and budget precede multiplication by group learning rates and exclude decoupled weight decay. They therefore do not guarantee task-loss descent or preservation.

B SPARSITY ACCOUNTING

B.1 EXACT COUNTERS

Activation sparsity at site s is $\mathcal{Z}_s = \sum_{\ell,e,i} \mathbf{1}\{A_{s,i}^{(\ell,e)} = 0\} / \sum_{\ell,e} |A_s^{(\ell,e)}|$, where ℓ indexes layers, e evaluation batches, i tensor entries, and $|A|$ the number of elements in tensor A . Near-zero mass replaces equality by $|A_{s,i}| \leq \varepsilon$ for a stated tolerance; it does not contribute to exact-zero counts unless the value is zero. All aggregate fractions pool integer numerators and denominators across batches and layers before division.

The atomic unit of model-wide accounting is one scalar multiplication in a matrix product. If $X \in \mathbb{R}^{n \times p}$ feeds a linear projection with q outputs, the count is $N = npq$, with $N_0 = q \sum_{i,j} \mathbf{1}\{X_{ij} = 0\}$. Zero weights receive no credit. For the multihead attention graph specified in Appendix C.1, both operands are activations, so a product is credited once if either is zero. Within one block and evaluation batch, for query position t , key position $u \leq t$, and within-head coordinate j ,

$$N_{0,QK} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{Q_{bhtj} = 0 \vee K_{bhuj} = 0\}, \quad (2)$$

$$N_{0,PV} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{P_{bhtu} = 0 \vee V_{bhuj} = 0\}. \quad (3)$$

Here P is the causal softmax probability tensor; the model numerator sums these counts over blocks and evaluation batches. Future masked positions are excluded from both counts, while zeros on valid positions, including floating-point underflow, are included. Counts use the operands actually consumed by each matrix product; upstream masks do not determine those operands' zeros after intervening transformations. Earlier value positions participate in more causal pairs, so local value sparsity alone does not determine PV sparsity.

B.2 SELECTED-SITE REACH CEILING AND NORMALIZATION

For a specified graph and workload, let N_o count the valid scalar multiplications in block operation family o , pooled over layers and examples. Then $N_{\text{model}} = \sum_o N_o + N_{\text{head}}$, while N_0 sums the corresponding zero-operand counts only in block operations. The block-only view is $\mathcal{S}_{\text{block}} = N_0 / \sum_o N_o$. Both sparsities are fractions; percentages multiply them by 100. Embedding lookup, normalization, nonlinearities, softmax, RoPE, bias and residual additions, and data movement are outside this accounting.

To construct the reach numerator, count products guaranteed to have a zero activation operand as a structural consequence of 100% activation sparsity at the selected sites, independently of checkpoint values or natural zeros elsewhere. Include exact zero-preserving propagation under the declared graph rules. Products reached through more than one site receive one credit. The resulting count is $N_{\text{reach}}(\mathcal{N})$. Division by the same N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$. Natural zeros outside this selected-site reach can contribute to observed $\mathcal{S}_{\text{model}}$, so the ceiling is not an upper bound on every observation. Its gap to an observation does not measure sparsity available without quality loss. Appendix C.2 specializes the denominator and reach rules to the Pythia graph used in the experiments.

C EXPERIMENTAL SETUP DETAILS

C.1 PYTHIA SITES AND NONLINEARITIES

Pythia uses parallel attention and FFN branches with separate LayerNorms. For batch size B , sequence length T , residual width d , FFN width d_f , and H heads, let $d_h = d/H$. Table 3 identifies the actual operands recorded after the selected nonlinearity. The same site mapping is applied in every layer, with one common threshold per condition. Figure 2 shows the shared block; Table 1 gives its thresholding and pressure assignments.

At h , the selected nonlinearity replaces GELU. Other sites retain their original activations unless a thresholding nonlinearity is applied. At zero threshold, A7's additional symmetric query/key/value nonlinearities are exact identities in both values and gradients. They therefore do not change A4's

Table 3: Intervention sites and the matrix products that consume them directly.

Site	Shape	Consuming operation	Placement
a	$B \times T \times d$	QKV projection	After attention LayerNorm
m	$B \times T \times d$	FFN up projection	After FFN LayerNorm
h	$B \times T \times d_f$	FFN down projection	Nonlinearity output
q, k	$B \times H \times T \times d_h$	Attention scores	After partial RoPE
v	$B \times H \times T \times d_h$	Probability-value	After the QKV split
z	$B \times T \times d$	Attention output	Concatenated context, before output projection

mathematical forward map in the absence of pressure. A7-OL1 still differs from A4-OL1 in its pressure targets and their equal-tensor normalization.

Query and key counts use actual post-RoPE operands: rotation can repopulate an individual zero coordinate. Likewise, the output projection uses actual post-nonlinearity context z , rather than inferring its zeros from the value mask.

C.2 ARCHITECTURE-SPECIFIC COUNTS AND SELECTED-SITE REACH

The operation inventory comprises QKV projection, attention scores (QK), probability-value products (PV), attention-output projection, and the two FFN projections. For one uncached, unpadded sequence, each block contributes $3Td^2$ products from QKV projection, Td^2 from the attention-output projection, Tdd_f from each FFN projection, and $dT(T+1)/2$ from each of QK and PV . Thus, for L blocks and vocabulary size V_{vocab} ,

$$\begin{aligned} C_{\text{block}} &= T(4d^2 + 2dd_f) + dT(T+1), \\ C_{\text{model}} &= LC_{\text{block}} + TdV_{\text{vocab}}. \end{aligned} \quad (4)$$

Over B sequences, $N_{\text{model}} = BC_{\text{model}}$; the dense head contributes only to the denominator, as defined in Section 3.2.

Let $\mathcal{O}(\mathcal{N})$ be the union of operations reachable when the selected sites have 100% activation sparsity: a reaches QKV; m reaches FFN up; h reaches FFN down; either q or k reaches QK ; v reaches PV and attention output; and z reaches attention output. Each operation is counted once. The v rule includes the exact closure $V = 0 \Rightarrow PV = 0$. No further propagation is assumed: QKV biases can survive $a = 0$, and zero scores do not imply zero softmax probabilities. For the same site set in every layer, $N_{\text{reach}} = BL \sum_{o \in \mathcal{O}(\mathcal{N})} C_o$, where C_o is that operation’s per-sequence count. Division by N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$.

With Pythia’s $d_f = 4d$, the per-block reachable counts for A0, A1-H, A4, and A7 are 0, $4Td^2$, $12Td^2$, and C_{block} , respectively. The ladder evaluates these formulas at $T = 2,048$, $V_{\text{vocab}} = 50,304$, and the pinned (L, d) configurations (6, 128), (6, 512), and (24, 1,024).¹ No checkpoint enters this calculation. For example, A0 has zero selected-site reach but may have positive observed sparsity from natural zeros.

Dividing the seven-site count by the full-model denominator gives

$$\mathcal{S}_{\text{model}}^{\text{max}}(\text{A7}) = \frac{L(12d + T + 1)}{L(12d + T + 1) + V_{\text{vocab}}}. \quad (5)$$

Greater width or depth reduces the dense head’s relative contribution under this workload. This is an analytic coverage fraction, not a prediction of learned sparsity at a given quality cost. It assumes full-vocabulary logits at every sequence position. Computing logits only at the final position would instead give denominator $LC_{\text{block}} + dV_{\text{vocab}}$, changing the reach reference even for the same architecture.

¹Architecture configurations: EleutherAI’s Pythia-14M-deduped, Pythia-70M-deduped, and Pythia-410M-deduped. Links identify immutable configuration revisions; released weights are not used.

C.3 EVALUATION SCOPE

Quality and logical counts are paired in the same eager-attention evaluation of a checkpoint, with its declared nonlinearities and evaluation precision. The complete MiniPile validation split contains 500 documents. Concatenation and packing produce 338 complete blocks of 2,048 tokens (692,224 input tokens); the final 1,444-token tail is excluded. Each block contributes 2,047 next-token targets to cross-entropy, while product accounting includes all 2,048 input positions. Loss is averaged over the equally sized blocks, and sparsity is count-pooled. This workload uses no key-value cache. Condition-specific training budgets, optimizer settings, seeds, and checkpoint selection accompany their results; they are not implied by the ladder’s list of candidate conditions.

C.4 REALIZED TRAINING PROTOCOL

We use MiniPile and the Pythia-14M-deduped tokenizer for every size. Each document is encoded without added special tokens, followed by one end-of-sequence token. Concatenated tokens are packed into nonoverlapping 2,048-token blocks. Training uses a seeded permutation of the block indices; validation retains source order. The training split contains 1,000,000 documents and 1,491,711,416 tokens, yielding 728,374 complete blocks and an excluded 1,464-token tail. The supplementary `protocol.json` records immutable dataset and tokenizer revisions, cache hashes and implementation identities.

All conditions start from random weights drawn from the pinned Pythia architecture configurations; released pretrained weights are not used. Model initialization and data-order seeds are both 1234, with one training seed per condition. Parameter sweeps measure intervention variability rather than variation over independent training seeds. All quality comparisons reuse the same validation split, without a separate confirmation set. Within a size, the conditions share the initial parameter hash and training schedule. The 30-condition 14M cohort contains A0, A1-H, four weights each for A1-H-L1 and A1-H-OL1, and five thresholds each for A4, A4-OL1, A7 and A7-OL1. Each larger cohort contains A0, A1-H, and the five-threshold A4-OL1/A7-OL1 sweeps. Local pressure weights are $\{0.05, 0.1, 0.5, 1\}$; thresholds are $\{0, 0.01, 0.05, 0.1, 0.5\}$. Multisite OL1 uses $\lambda = 1$, with pressure at all four or seven selected sites. Every OL1 condition uses trust budget 1 and projection stabilizer $\epsilon = 10^{-12}$.

Training uses AdamW with $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.1 and global task-gradient clipping at norm 1. Naive L1 clips the combined objective gradient instead. The learning rate warms up linearly for eight updates and follows cosine decay to one tenth of its peak. Parameters and optimizer state are FP32; forward computation uses FP16 autocast with dynamic loss scaling. Attention and hidden dropout are zero. Table 4 gives size-dependent settings. The global batch is 1,024 sequences of 2,048 tokens, for 712 optimizer updates and 1,493,172,224 input tokens. This wraps 714 blocks beyond the 728,374 complete blocks in one packed training pass. We use the final checkpoint without validation-based checkpoint selection. The lower 410M learning rate is part of the realized protocol, so size alone is not the only cross-scale change.

Table 4: Size-dependent training settings. Tokens per parameter use the common realized input-token budget; they are not held fixed across sizes.

Setting	14M	70M	410M
Parameters	14,067,712	70,426,624	405,334,016
Peak learning rate	10^{-3}	10^{-3}	3×10^{-4}
Minimum learning rate	10^{-4}	10^{-4}	3×10^{-5}
Microbatch (sequences)	32	4	4
Gradient accumulation	32	256	256
Tokens per parameter	106.14	21.20	3.68

Figure 8 shows the A0 optimization trajectories under the common token budget. Training loss trends downward across the runs, but the three sizes finish in different optimization regimes: 70M reaches the lowest training loss under the realized schedule, while 410M finishes above it and exhibits a larger pre-clipping task-gradient norm late in training. These observations are **consistent with different degrees of optimization under the fixed-token protocol**, but they do not establish

whether any run has or has not converged. The trajectories therefore provide context for the non-monotonic A0 validation losses in Figure 6 rather than a basis for extrapolating performance with additional training.

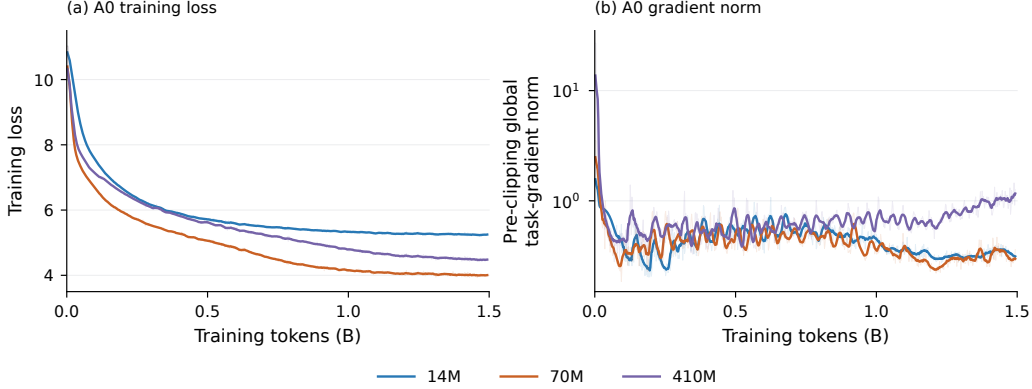


Figure 8: **A0 optimization trajectories under the common training-token budget.** (a) Training loss as a function of consumed input tokens for the 14M, 70M, and 410M A0 runs. (b) Pre-clipping global task-gradient norm over the same token budget; the logarithmic y-axis shows the large initial gradient transient and subsequent training regime. Faint traces show all 712 logged optimizer steps per size; opaque curves show a centered nine-step arithmetic moving mean, with partial windows at the edges. Norms are measured after loss unscaling and gradient accumulation, before clipping; smoothing is applied in the original units. All three sizes receive approximately 1.493B input tokens, but this corresponds to progressively fewer tokens per parameter as model size increases; the 410M run also uses a lower peak learning rate. Global norms are not normalized by parameter count. These trajectories characterize the realized optimization regimes and are not used as a convergence criterion.

D COMPLETE RESULTS AND SUPPORTING DIAGNOSTICS

The following tables report every selected trained endpoint, all 29 paired 14M contrasts and all 15 cross-scale recipe comparisons. They retain dominated settings as well as favorable ones. Each entry summarizes one trained model on complete validation. Sparsity percentages are formed from pooled integer product counts. The endpoint table uses the common A7 sparsity-ceiling denominator used for the endpoint annotations in Figure 6, rather than a different denominator for each recipe. Here $U_{\text{arch}} = \mathcal{S}_{\text{model}} / \mathcal{S}_{\text{model}}^{\text{max}}(\text{A7})$ denotes this common-reference ratio, retained for the complete endpoint table. Natural zeros outside the selected sites can make this ratio exceed one; it is not a universally bounded utilization fraction.

D.1 ACTIVATION SPARSITY AND SIGNED-DENSITY COVERAGE

The seven frozen-checkpoint evaluations in Figure 9 record signed values after the trained nonlinearities, with Q/K measured after RoPE. Each checkpoint contributes 2,658,140,160 FFN elements and 1,594,884,096 attention elements across the complete validation set. Summing integer counts gives the 80:20 weighting of $h : m$ and equal weighting of $q : k : v$. These pooled distributions are element-weighted mixtures, not distributions of layer means.

Exact zeros are stored separately from the nonzero histogram. The native grid spans $[-8, 8]$ with 16,000 bins of nominal width 0.001; the figure combines ten bins at a time and divides by the actual floating-point bin width and the count of all captured elements. Consequently, a curve’s full integral is the recorded nonzero mass inside that grid, not one. Table 8 reports the zero mass and the fractions outside the displayed and native ranges. Underflow and overflow counts are retained rather than folded into edge bins. The displayed limits are $[-2.5, 3]$ for FFN and $[-4, 4]$ for attention in Figure 9. The largest omitted display tail is 6.6719%; the native-grid tail is at most 0.7607%. Figure 5 separates m and h for the dense reference and the two OL1 checkpoints at $\kappa = 0.5$, using

Table 5: All 54 trained endpoints. λ is pressure weight; κ is the trained threshold. $U_{\text{arch}} = \mathcal{S}_{\text{model}}/\mathcal{S}_{\text{model}}^{\text{max}}$ (A7) uses the same A7 reference within each model size, including for A0. Rounded zero sparsity need not be exactly zero.

Size	Recipe	Parameter	Loss	$\mathcal{S}_{\text{model}}$ (%)	U_{arch} (%)
14M	A0	—	5.2086	0.000	0.000
14M	A1-H	—	5.2696	2.714	9.061
14M	A1-H-L1	$\lambda = 0.05$	5.2061	3.142	10.489
14M	A1-H-L1	$\lambda = 0.1$	5.1655	3.336	11.139
14M	A1-H-L1	$\lambda = 0.5$	5.1127	3.788	12.645
14M	A1-H-L1	$\lambda = 1$	5.1023	3.949	13.185
14M	A1-H-OL1	$\lambda = 0.05$	5.1981	3.145	10.501
14M	A1-H-OL1	$\lambda = 0.1$	5.1594	3.339	11.146
14M	A1-H-OL1	$\lambda = 0.5$	5.1102	3.768	12.580
14M	A1-H-OL1	$\lambda = 1$	5.1212	3.938	13.149
14M	A4	$\kappa = 0$	5.4705	7.212	24.078
14M	A4	$\kappa = 0.01$	5.4665	7.414	24.752
14M	A4	$\kappa = 0.05$	5.4341	8.206	27.396
14M	A4	$\kappa = 0.1$	5.4197	8.953	29.891
14M	A4	$\kappa = 0.5$	5.6597	10.216	34.106
14M	A4-OL1	$\kappa = 0$	5.4583	7.810	26.075
14M	A4-OL1	$\kappa = 0.01$	5.4583	8.587	28.668
14M	A4-OL1	$\kappa = 0.05$	5.4898	10.456	34.910
14M	A4-OL1	$\kappa = 0.1$	5.5483	11.394	38.041
14M	A4-OL1	$\kappa = 0.5$	6.0380	12.713	42.446
14M	A7	$\kappa = 0$	5.4684	7.218	24.097
14M	A7	$\kappa = 0.01$	5.4588	7.618	25.433
14M	A7	$\kappa = 0.05$	5.4379	9.127	30.471
14M	A7	$\kappa = 0.1$	5.4287	10.425	34.805
14M	A7	$\kappa = 0.5$	5.7029	15.387	51.371
14M	A7-OL1	$\kappa = 0$	5.4802	7.054	23.551
14M	A7-OL1	$\kappa = 0.01$	5.4758	7.723	25.786
14M	A7-OL1	$\kappa = 0.05$	5.4628	9.863	32.929
14M	A7-OL1	$\kappa = 0.1$	5.4295	11.797	39.385
14M	A7-OL1	$\kappa = 0.5$	5.8294	27.483	91.755
70M	A0	—	4.0998	0.000	0.001
70M	A1-H	—	4.2227	10.066	20.366
70M	A4-OL1	$\kappa = 0$	4.8054	25.673	51.944
70M	A4-OL1	$\kappa = 0.01$	4.8229	26.224	53.060
70M	A4-OL1	$\kappa = 0.05$	4.8742	28.490	57.644
70M	A4-OL1	$\kappa = 0.1$	4.9439	30.574	61.861
70M	A4-OL1	$\kappa = 0.5$	5.3895	35.596	72.022
70M	A7-OL1	$\kappa = 0$	4.9412	23.562	47.674
70M	A7-OL1	$\kappa = 0.01$	4.9364	24.226	49.017
70M	A7-OL1	$\kappa = 0.05$	4.9637	26.524	53.666
70M	A7-OL1	$\kappa = 0.1$	4.9501	28.694	58.056
70M	A7-OL1	$\kappa = 0.5$	5.2159	40.602	82.150
410M	A0	—	4.5475	0.011	0.013
410M	A1-H	—	4.6513	21.050	24.127
410M	A4-OL1	$\kappa = 0$	5.6921	38.339	43.944
410M	A4-OL1	$\kappa = 0.01$	5.6914	39.149	44.873
410M	A4-OL1	$\kappa = 0.05$	5.6789	42.125	48.283
410M	A4-OL1	$\kappa = 0.1$	5.6789	45.688	52.367
410M	A4-OL1	$\kappa = 0.5$	5.1910	71.591	82.058
410M	A7-OL1	$\kappa = 0$	5.4263	41.121	47.133
410M	A7-OL1	$\kappa = 0.01$	5.4331	41.478	47.542
410M	A7-OL1	$\kappa = 0.05$	5.4679	44.053	50.494
410M	A7-OL1	$\kappa = 0.1$	5.4907	48.998	56.161
410M	A7-OL1	$\kappa = 0.5$	5.1207	80.616	92.401

Table 6: All 29 Pythia-14M contrasts, treatment minus reference. When both recipes have a swept parameter, its value is matched; A0 and A1-H are fixed controls.

Treatment – reference	Parameter	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
A1-H – A0	—	+0.0611	+2.714
A1-H-L1 – A1-H	$\lambda = 0.05$	-0.0635	+0.427
A1-H-L1 – A1-H	$\lambda = 0.1$	-0.1042	+0.622
A1-H-L1 – A1-H	$\lambda = 0.5$	-0.1569	+1.073
A1-H-L1 – A1-H	$\lambda = 1$	-0.1674	+1.235
A1-H-OL1 – A1-H-L1	$\lambda = 0.05$	-0.0081	+0.004
A1-H-OL1 – A1-H-L1	$\lambda = 0.1$	-0.0061	+0.002
A1-H-OL1 – A1-H-L1	$\lambda = 0.5$	-0.0025	-0.020
A1-H-OL1 – A1-H-L1	$\lambda = 1$	+0.0189	-0.011
A4 – A1-H	$\kappa = 0$	+0.2009	+4.498
A4 – A1-H	$\kappa = 0.01$	+0.1969	+4.700
A4 – A1-H	$\kappa = 0.05$	+0.1645	+5.492
A4 – A1-H	$\kappa = 0.1$	+0.1500	+6.239
A4 – A1-H	$\kappa = 0.5$	+0.3900	+7.501
A4-OL1 – A4	$\kappa = 0$	-0.0122	+0.598
A4-OL1 – A4	$\kappa = 0.01$	-0.0082	+1.173
A4-OL1 – A4	$\kappa = 0.05$	+0.0556	+2.251
A4-OL1 – A4	$\kappa = 0.1$	+0.1286	+2.441
A4-OL1 – A4	$\kappa = 0.5$	+0.3783	+2.498
A7 – A4	$\kappa = 0$	-0.0021	+0.006
A7 – A4	$\kappa = 0.01$	-0.0077	+0.204
A7 – A4	$\kappa = 0.05$	+0.0038	+0.921
A7 – A4	$\kappa = 0.1$	+0.0090	+1.472
A7 – A4	$\kappa = 0.5$	+0.0432	+5.171
A7-OL1 – A7	$\kappa = 0$	+0.0118	-0.163
A7-OL1 – A7	$\kappa = 0.01$	+0.0170	+0.106
A7-OL1 – A7	$\kappa = 0.05$	+0.0249	+0.736
A7-OL1 – A7	$\kappa = 0.1$	+0.0008	+1.372
A7-OL1 – A7	$\kappa = 0.5$	+0.1265	+12.096

Table 7: All matched A7-OL1 minus A4-OL1 comparisons. Negative loss changes and positive sparsity changes favor A7-OL1.

Size	κ	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
14M	0	+0.0219	-0.756
14M	0.01	+0.0175	-0.863
14M	0.05	-0.0270	-0.593
14M	0.1	-0.1188	+0.402
14M	0.5	-0.2086	+14.769
70M	0	+0.1358	-2.110
70M	0.01	+0.1135	-1.998
70M	0.05	+0.0895	-1.966
70M	0.1	+0.0062	-1.880
70M	0.5	-0.1736	+5.006
410M	0	-0.2658	+2.783
410M	0.01	-0.2582	+2.329
410M	0.05	-0.2111	+1.929
410M	0.1	-0.1881	+3.310
410M	0.5	-0.0703	+9.024

the same density normalization and symlog scale. Its limits are $[-2.5, 3]$ for each FFN site and $[-4, 4]$ for pooled Q/K/V. The zero percentages use measured site counts, not inversion of rounded operation contributions. Pressure-free histograms were not retained; their operation counts support the main figure’s bars.

The diagnostic reruns use FP32 parameters with FP16 autocast and no additional thresholding. Their maximum absolute validation-loss difference from the retained reference is 6.89×10^{-5} , below the specified 5×10^{-4} tolerance; all 108 threshold-consistency checks pass. Small rounding differences in exact-zero counts between evaluation passes do not change the displayed conclusions. Complete per-site and pooled histograms, counts and checkpoint hashes accompany the data release.

Table 8: Activation sparsity (the probability mass at zero) omitted from the density curves, and mass outside their displayed range (view tail) and the native $[-8, 8]$ histogram (grid tail). Grid tails are a subset of view tails, not an additional mass to add. All percentages divide integer counts by all captured elements. A0 is listed once although it is repeated in each figure row. These 14 groups come from seven checkpoints.

Recipe	κ	Group	Sparsity (%)	View tail (%)	Grid tail (%)
A0	—	FFN	< 0.01	0.1113	0.0000
A0	—	Attention	< 0.01	2.3929	0.4632
A4-OL1	0	FFN	67.22	0.0003	0.0000
A4-OL1	0	Attention	< 0.01	5.1717	0.6162
A4-OL1	0.05	FFN	86.71	0.0003	0.0000
A4-OL1	0.05	Attention	< 0.01	5.6601	0.7607
A4-OL1	0.5	FFN	99.31	0.0001	0.0000
A4-OL1	0.5	Attention	0.22	0.0003	0.0000
A7-OL1	0	FFN	64.52	0.0063	0.0000
A7-OL1	0	Attention	< 0.01	5.4657	0.4648
A7-OL1	0.05	FFN	75.29	0.0106	0.0000
A7-OL1	0.05	Attention	7.26	6.6719	0.6932
A7-OL1	0.5	FFN	93.64	0.0037	0.0000
A7-OL1	0.5	Attention	95.60	0.2396	0.0009

How interventions reshape activation distributions (Pythia-14M)

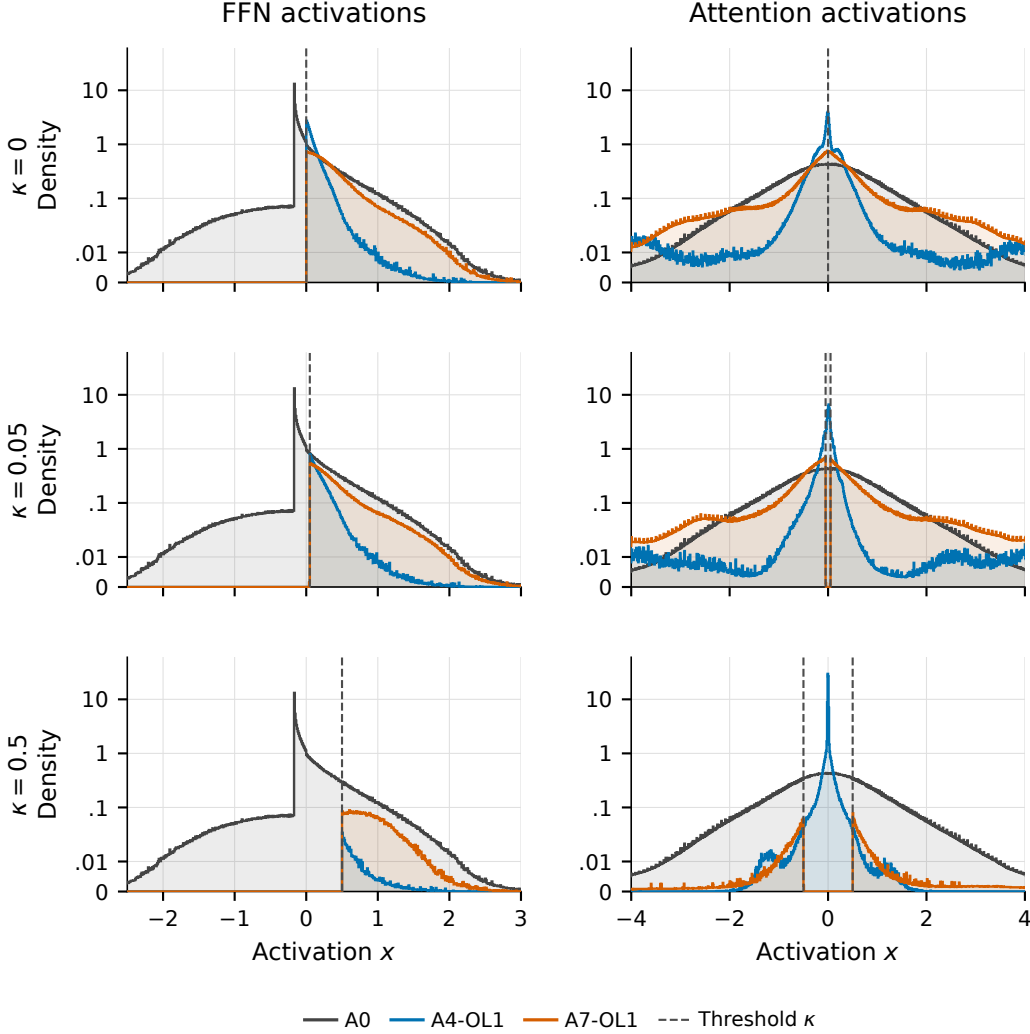


Figure 9: **Interventions reshape FFN and attention distributions differently. Exact zeros are omitted from the curves and reported in Table 8.** Columns pool FFN (h, m) and attention (q, k, v) activations; rows use trained thresholds $\kappa = 0, 0.05, 0.5$. A0 is repeated across rows. Counts are pooled across all layers and validation blocks, weighting $h : m$ by 80:20 and $q : k : v$ equally. Density divides nonzero bin counts by all captured elements and bin width, without renormalizing the visible range. The vertical scale is symlog, linear below 0.01; filled areas on this scale do not represent probability mass. Dashed lines mark $+\kappa$ for A4/A7 FFN thresholds and $\pm\kappa$ for A7 attention thresholds. Q/K are measured after RoPE. A4 leaves q, k, v without direct thresholding or pressure; A7 targets them directly. The seven checkpoints use no additional post-hoc magnitude thresholding.

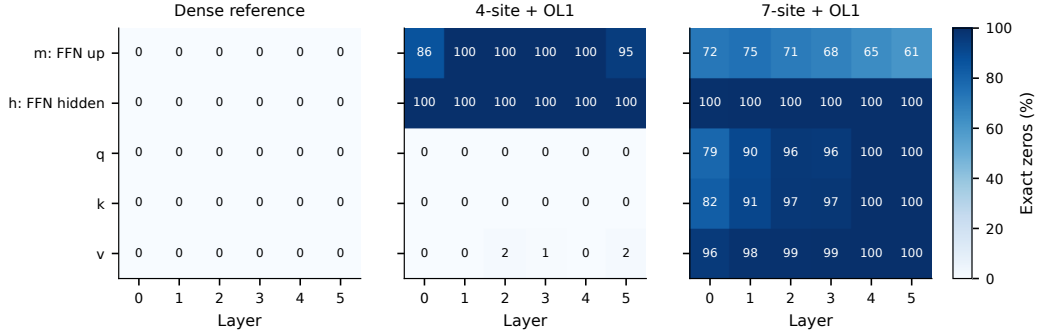


Figure 10: **Exact-zero fractions by activation site and layer.** Counts pool all 338 validation sequences within each cell, for the dense reference and four-/seven-site OL1 at $\kappa = 0.5$. Q/K use post-RoPE operands. The common scale is 0–100%; displayed cell values are rounded to whole percentages, so a displayed 100 does not imply every entry is zero. The FFN-up input m changes substantially between the OL1 recipes, while hidden activation h remains nearly entirely zero. These observations do not isolate thresholding, pressure normalization or cap behavior.

D.2 OPERATION CONTRIBUTIONS TO MODEL-WIDE SPARSITY

An operation’s contribution to S_{model} is its zero-product count divided by the full model product count. Summing these contributions recovers the model-wide fraction; summing local activation percentages would not. Table 9 gives this decomposition at $\kappa = 0.5$. At 14M, extending from A4-OL1 to A7-OL1 adds attention opportunity while reducing the contribution from some projections. At larger sizes, projection work carries more weight in the workload, so an equal change in activation sparsity need not yield an equal change in model-wide sparsity. Figure 11 displays the operation-level view at the same threshold and all three sizes.

Table 9: Contributions to $100S_{\text{model}}$ in percentage points at $\kappa = 0.5$. All six operations share the same model denominator within each size. The dense LM head contributes to that denominator but receives no zero credit. Differences in rounded entries can affect the displayed sum.

Operation	14M		70M		410M	
	A4-OL1	A7-OL1	A4-OL1	A7-OL1	A4-OL1	A7-OL1
QKV	3.173	2.430	8.820	6.231	17.563	16.992
QK	0.000	8.324	0.000	5.052	0.000	5.206
PV	0.057	8.449	0.011	5.371	0.008	5.858
Attention output	1.069	1.069	3.087	3.084	6.186	6.187
FFN up	4.139	2.939	11.329	8.526	22.938	21.503
FFN down	4.275	4.272	12.350	12.338	24.895	24.869
Total	12.713	27.483	35.596	40.602	71.591	80.616

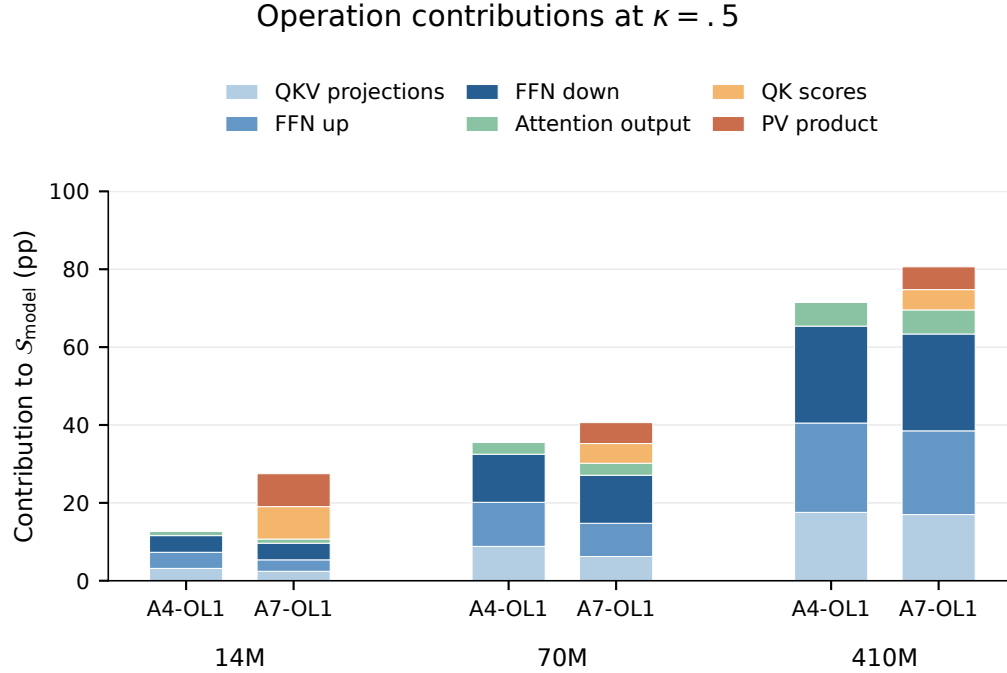


Figure 11: Operation accounting for A4-OL1 and A7-OL1 at $\kappa = 0.5$ across 14M, 70M and 410M. Zero-operand products are counted over causally valid positions and pooled across all 338 validation blocks. Each contribution uses the full model denominator, including the dense LM head without zero credit. At 14M, attention contributes more under A7-OL1, outweighing smaller contributions from several projections. The bars show logical multiplication opportunities, not runtime savings.

D.3 COMPLETE POST-HOC MAGNITUDE THRESHOLDING TRAJECTORIES

The retained evaluation set contains 540 evaluations: ten threshold targets for each of the 54 trained checkpoints. It joins 350 new evaluations with 190 retained evaluations matched by checkpoint content hash. Calibration uses the first ten complete training blocks in source order, separately for each layer’s a, m, h, z activations. Evaluation zeros values satisfying $|x| \leq t$ after the trained nonlinearities, without changing weights or A7’s internal attention nonlinearities. Historical figure labels use “clipping” for this magnitude-thresholding operation; amplitudes above the threshold are unchanged. The target p controls a calibration quantile; it is not the achieved model-wide sparsity. In particular, an existing zero mass can make several targets identical.

Figures 12–14 retain the full loss range and every target, including dominated points and ties. Each path begins at its actually measured $p = 0$ evaluation. Small numerical differences from the canonical training endpoint can change strict nondominance without implying a substantive improvement. Numerical frontier memberships are stored separately from the lines connecting evaluated settings. All evaluations use complete validation with FP32 parameters, FP16 autocast, eager attention, batch one and $T = 2,048$.

14M training and post-hoc clipping

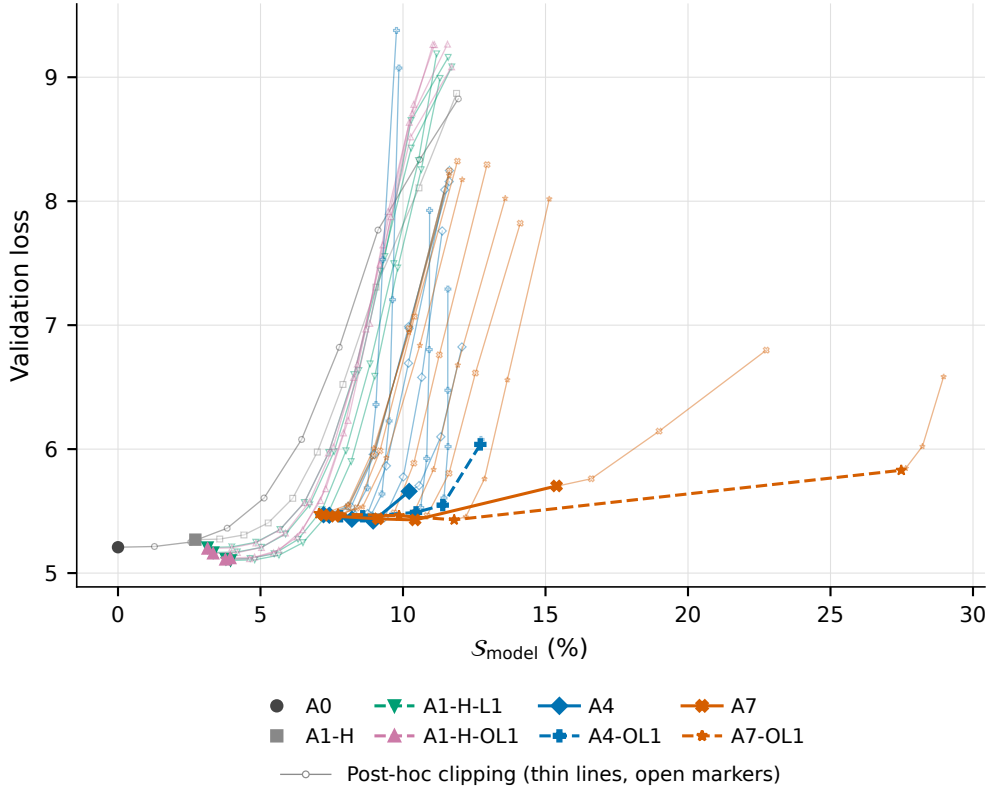


Figure 12: Complete 14M training and post-hoc magnitude thresholding results: 30 trained endpoints and 300 thresholding evaluations. Large filled markers denote trained endpoints; bold lines connect their settings, solid for unpressed recipes and dashed for L1/OL1. Each faint open-marker path follows one fixed checkpoint through $p = 0, 0.1, \dots, 0.9$. Colors and symbols identify its source recipe. The full loss range is shown. Connections order evaluated settings and do not interpolate attainable models or fitted Pareto envelopes.

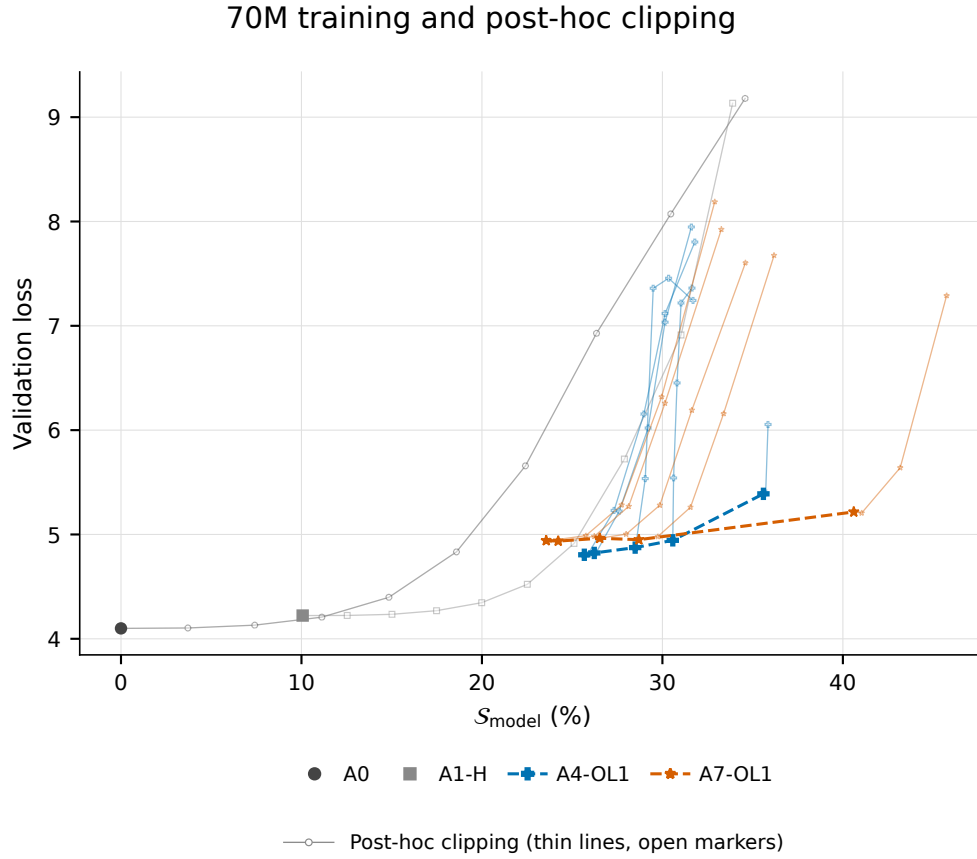


Figure 13: Complete 70M training and post-hoc magnitude thresholding results: 12 trained end-points and 120 thresholding evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing threshold targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 12.

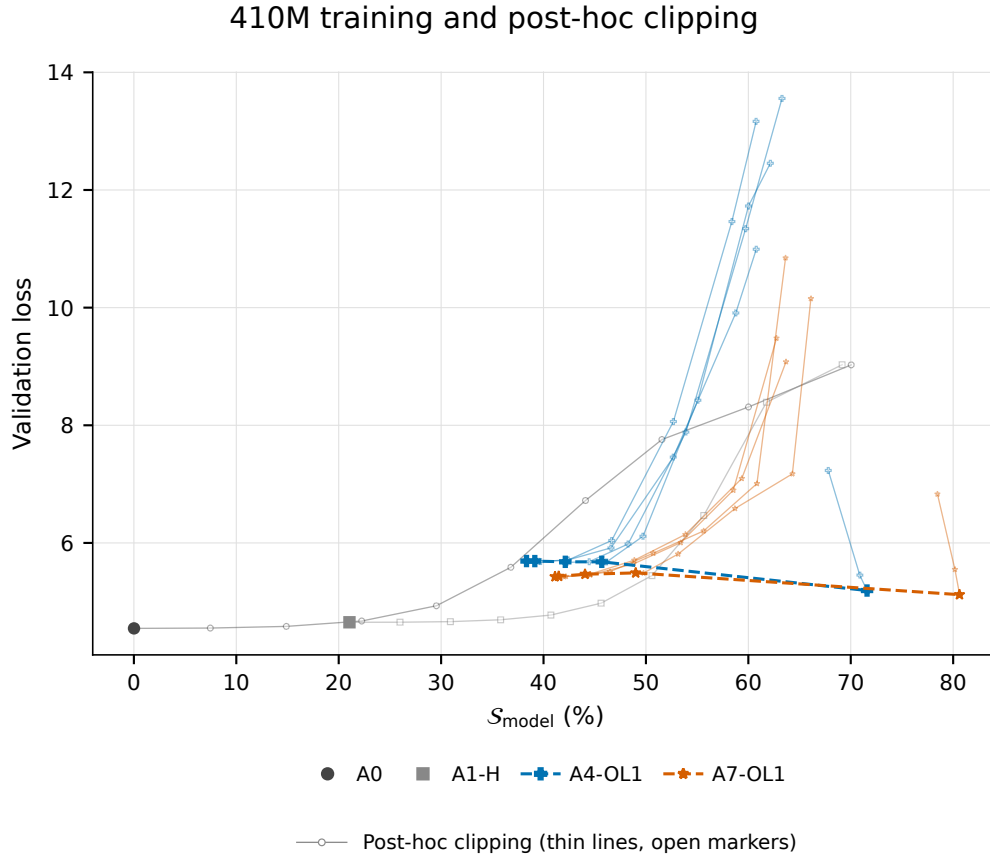


Figure 14: Complete 410M training and post-hoc magnitude thresholding results: 12 trained end-points and 120 thresholding evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing threshold targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 12.

D.4 KERNEL QUALIFICATION AND ABLATIONS

The unchanged Sakana/TwELL control achieves $1.3001\times$ on its supported SparseLM-0.5B/H100 workload. Its raw projection primitive at the Pythia-14M width $N = 128$ instead gives relative L2 error 0.69055, exceeding the 0.02 bound; that test stops before full-model Pythia timing. This is a compatibility test, not an unchanged upstream speedup comparison on our model. The later P0 adapter changes signed-value packing, shape tails and numerical handling for Pythia. It retains dense attention and uses native projection fallback on A0. Table 10 reports this adapted baseline.

Historical proposals are replayed on four representative checkpoints; the incumbent in Figure 15 uses fixed A7-OL1, $\kappa = 0.5$. It reaches $1.6024\times$ by iteration 11 and $1.7830\times$ at iteration 42. Final implementation and ablation summaries retain the 30-checkpoint manuscript cohort from the historical 35-checkpoint sweep. Five A4 checkpoints with pressure only at h are excluded because they do not implement the four-site pressure objective studied here. Search progress and the final sweep are separate measurements, explaining the small difference between their $1.7830\times$ and $1.7832\times$ maxima.

Sparse kernel auto-research for full-model acceleration

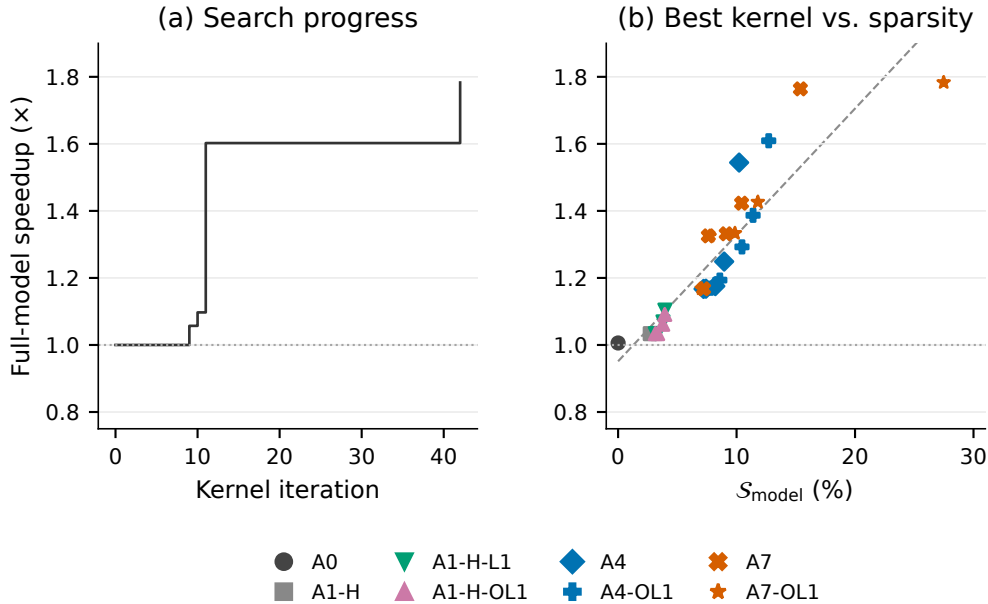


Figure 15: **Kernel-search history and the final checkpoint sweep.** (a) Best qualified incumbent across 42 eligible retrospective proposals on the fixed seven-site + OL1, $\kappa = 0.5$ checkpoint. (b) Final K050 speedup across the 30-checkpoint cohort; this historical view distinguishes individual recipes. Each ratio uses that checkpoint’s own native PyTorch/SDPA CUDA-graph latency, not a common dense-model reference. All use the same BF16, batch-size-one, 2,048-token protocol on one RTX5090. Search replay and final evaluation are separate timing measurements. The history describes one human-guided development trajectory, not a comparison of coding agents.

The coding-agent configuration records *gpt-6-astra* as the default model, not the model served for every request. This is one human-guided development trajectory and does not compare agent-assisted with human-only kernel development. Kernel measurements use Python 3.12.3, PyTorch 2.11.0+cu128, Transformers 5.12.1, CUDA 12.8 and nvcc 12.8.93. The supplementary protocol records dependency and frozen-source hashes.

Every candidate must produce finite outputs, with each logit satisfying $|y - \hat{y}| \leq 0.25 + 0.02|\hat{y}|$, per-block relative output L2 error at most 0.02, and absolute pooled loss difference at most 0.001. Here $\hat{y}$ is the unmodified native eager reference. All three fresh processes must pass on all 338 validation blocks. Qualification is bounded numerical agreement on these inputs, not bitwise equivalence.

Failed candidates are excluded from qualified speedup summaries. P0, an early projection adapter derived from the released kernels of Cetin et al. (2026), qualifies on only four checkpoints. Its average is not comparable to a 30-checkpoint mean or an unchanged upstream benchmark.

Timing uses the same checkpoint, inputs and CUDA-graph scaffold for native PyTorch/SDPA and each candidate. For each checkpoint, we take the geometric mean of 1,344 paired host-latency ratios (native divided by candidate): 64 inputs, seven randomized paired passes, and three fresh processes. Compilation, static weight preparation, graph capture and equal input staging are excluded. Preprocessing that depends on activations, embeddings, every block, final normalization and full vocabulary logits are included. There is no activation cache. Cohort summaries give each checkpoint equal weight.

K050 combines normalization and RoPE fusion, zero-fragment tensor-core bypass and sparse-row projection paths, using CUDA, CUTLASS, FlashAttention and PyTorch components. K049 is its searched predecessor; K050 adds fusion of the two input LayerNorms while preserving their separate affine parameters and thresholds. The all-skips-off control retains fusion while disabling both attention and projection skips; the attention-dense control retains projection skips. The historical ablation summary forms ratios from separately measured, native-normalized timings. Figure 7b instead directly divides candidate latency geometric means, as defined below. K050’s sparse paths improve 14 of 30 checkpoints relative to all-skips-off, with geometric mean ratio $1.0432\times$. Disabling attention skipping improves all 30: at the search checkpoint, instrumentation confirms that skipping bypasses 58.0% of eligible QK and 67.2% of PV tensor-core matrix multiply-accumulate (MMA) instructions, yet its overhead exceeds the saved time. Scalar zero-product fractions and skipped MMA fractions are different estimands.

The attention path tests whether either warp-distributed operand fragment contains only zeros, and issues an MMA only when both fragments contain a nonzero. Partial sparsity can therefore leave an entire instruction active. Zero detection, warp votes and branches recur inside the kernel; causal masking and softmax still execute. A zero QK score does not imply a zero softmax probability. The measurements do not identify a profitable joint-zero pattern or break-even threshold. They show that the skipped work does not repay these costs for the measured 14M shapes and hardware.

Table 10: Numerically qualified full-model speedups against native SDPA CUDA graphs. GM is the equal-checkpoint geometric mean; the range is across qualified checkpoints, not a confidence interval. All conditions use one RTX5090, BF16, batch one, 2,048 tokens and full vocabulary logits. P0’s four-checkpoint subset is shown only to document its limited qualification.

Implementation	Qualified	GM speedup	Range
K049	30/30	1.1141	0.9550–1.5284
K050	30/30	1.2340	1.0059–1.7832
K050, attention dense	30/30	1.2506	1.0185–1.8022
K050, all skips off	30/30	1.1829	1.0503–1.3628
P0 (qualified subset only)	4/30	0.8910	0.8180–1.0012

D.5 ZERO-FRAGMENT STRUCTURE AND MATCHED RUNTIME DIAGNOSTICS

We reconcile the same 30 checkpoints with their canonical logical-product counts, retained kernel diagnostics, and raw matched timing pairs. Table 11 separates native and optimized latencies for each checkpoint. Let t_N, t_0, t_P, t_K denote the geometric-mean latencies of native execution, all-skips-off K050, projection-only K050, and full K050. The gains from enabling projection and attention skipping are t_0/t_P and t_P/t_K , respectively. Both controls retain K050’s fusion; the former disables attention skipping in both implementations. These are full-model latency ratios, not timings of the projection operations alone. t_N/t_0 compares native execution with the optimized all-skips-off control, which also changes dense kernels and layouts, so it does not isolate fusion. Native-relative full speedup is t_N/t_K ; t_N uses the native measurements paired with full K050. Each geometric mean uses 64 inputs, seven paired passes, and three processes, matching the original timing protocol.

A common dense-reference ratio instead uses $t_{N,A0}/t_K$ for every checkpoint, with $t_{N,A0} = 0.6554424$ ms. At $\kappa = 0.5$, four-/seven-site OL1 then reach $1.427\times$ and $1.385\times$, respectively. This common denominator avoids rewarding recipe-specific native overhead, but still combines im-

plementation changes and different quality operating points. It is a ratio of retained geometric means across models, not a new interleaved timing comparison. The original native-relative associations below describe implementation acceleration, not a ranking of optimized recipe latency.

Execution units. QKV and FFN-up omit a $16 \times 8 \times 16$ MMA when the entire 16-token by 16-feature activation fragment is zero. Their counters are reconstructed from the actual BF16 operands using the implemented predicate. FFN-down and attention output process eight real token rows per padded 16-row MMA. Rows with at most two nonzeros may use scalar operations under the kernel’s numerical safety checks; remaining empty feature fragments bypass MMAs. These latter counters are instrumented and independently checked against operand-derived counts. Their bypass totals include replacement by scalar work as well as empty fragments, so they are not counts of eliminated arithmetic. We pool bypassed and eligible MMA counts across the four projection families before dividing, retaining the padding and scalar-work distinctions in the supplementary data.

Attention omits an MMA when either complete operand fragment is zero. In 29 checkpoints, QK skips 0% and PV skips 10.42% of eligible instructions; the PV baseline includes masked or padded probability work. Only seven-site + OL1 at $\kappa = 0.5$ has additional skipping, reaching 58.00% for QK and 67.21% for PV. Attention-skipping gain nevertheless remains below one for every checkpoint ($0.9832\text{--}0.9894\times$). These instruction counts include tiled causal padding, unlike the unmasked scalar logical-product denominator.

Associations and matched recipes. Figures 16–17 separate within-family associations from scalar and instruction-level predictors. Projection-skipping gain has Pearson $r = 0.973$, $R^2 = 0.946$ against projection MMA bypass, versus $r = 0.643$, $R^2 = 0.413$ against the projection scalar zero-product fraction. The within-family $\mathcal{S}_{\text{model}}$ versus native-relative speedup fits have $R^2 = 0.542$, 0.784, and 0.762 for the baseline/one-site, four-site, and seven-site groups (ten checkpoints each). Centering both variables within family gives $R^2 = 0.660$. The global relation is therefore not solely separation between family means; all fits remain descriptive and do not control for threshold, pressure, quality, or native cost. Attention MMA skipping has only two distinct predictor values, with one checkpoint at the larger value, limiting its correlation to that contrast.

Across all five matched OL1 thresholds, four-site recipes have greater projection scalar contribution, more projection MMA bypass, and larger projection-skipping gains than seven-site recipes. Without pressure, small bypass differences change sign, although four-site has the larger gain at each threshold. At $\kappa = 0.5$ with OL1, four-site bypasses 82.30% of projection MMAs versus 57.68% for seven-site. Its full K050 latency is also lower: 0.45948 versus 0.47337 ms. Seven-site nevertheless reports the larger native-relative speedup ($1.783\times$ versus $1.609\times$), because its native baseline is slower (0.84409 versus 0.73944 ms). Figure 18 displays this difference. The relative-speedup/absolute-latency reversal holds in all eight positive-threshold four-/seven-site pairs, with and without OL1.

The native implementation executes Q/K/V gates separately at nonzero thresholds; K050 fuses them with layout conversion and RoPE, including in the all-skips-off control. Similar all-skips-off latencies support native gate overhead as an explanation for the denominator difference, but do not isolate each operation’s timing contribution.

Coverage and unavailable quantities. Logical sparsity uses retained FP16 counts. Kernel counters use actual BF16 operands over all 338 validation blocks; timing uses the original 64-input subset. Pure-zero-only versus scalar-replacement bypass counts, counters restricted to the timed inputs, unmasked-only attention MMA counts, and isolated per-operation latency attribution are unavailable. No such values are inferred from $\mathcal{S}_{\text{model}}$. Supplementary `kernel/checkpoints.csv` retains all 30 records with operation counts, skip counters, latencies, and explicit missing fields; `kernel/associations.csv` and `kernel/matched-pairs.csv` retain the complete fits and matched contrasts.

Table 11: **Matched K050 latency ablations for every 14M checkpoint.** Entries are geometric-mean full-model latencies in milliseconds. Native uses the reference paired with full K050. All-skips-off retains the optimized implementation but disables both sparse paths; projection-only enables projection skipping with attention skipping disabled. The dose is λ for local L1/OL1 and κ for four-/seven-site recipes; multisite OL1 fixes $\lambda = 1$. Ratios should be computed from the unrounded supplementary values. Differences across checkpoints are not independent-seed estimates.

ID	Recipe	λ/κ	Native	All skips off	Proj. only	Full K050
c01	A0	–	0.65544	0.62147	0.64373	0.65157
c02	A1-H	–	0.65417	0.62230	0.62420	0.63282
c03	A1-H-L1	0.05	0.65435	0.62190	0.62375	0.63302
c04	A1-H-L1	0.1	0.65422	0.62238	0.62408	0.63251
c05	A1-H-L1	0.5	0.65447	0.62209	0.60448	0.61226
c06	A1-H-L1	1	0.65455	0.62296	0.58428	0.59282
c07	A1-H-OL1	0.05	0.65413	0.62234	0.62557	0.63256
c08	A1-H-OL1	0.1	0.65446	0.62224	0.62467	0.63308
c09	A1-H-OL1	0.5	0.65438	0.62290	0.60831	0.61633
c10	A1-H-OL1	1	0.65426	0.62181	0.59167	0.59960
c11	A4	0	0.73993	0.62327	0.62495	0.63391
c12	A4	0.01	0.74030	0.62344	0.62535	0.63391
c13	A4	0.05	0.74012	0.62313	0.62148	0.62958
c14	A4	0.1	0.73999	0.62294	0.58446	0.59255
c15	A4	0.5	0.73940	0.62389	0.47116	0.47882
c16	A4-OL1	0	0.74048	0.62294	0.62479	0.63328
c17	A4-OL1	0.01	0.74015	0.62394	0.61189	0.62014
c18	A4-OL1	0.05	0.74025	0.62340	0.56503	0.57268
c19	A4-OL1	0.1	0.74017	0.62296	0.52610	0.53371
c20	A4-OL1	0.5	0.73944	0.62263	0.45175	0.45948
c21	A7	0	0.74015	0.62266	0.62605	0.63396
c22	A7	0.01	0.84616	0.62117	0.63061	0.63835
c23	A7	0.05	0.84608	0.62064	0.62669	0.63533
c24	A7	0.1	0.84553	0.62065	0.58647	0.59417
c25	A7	0.5	0.84523	0.62121	0.47200	0.47914
c26	A7-OL1	0	0.74032	0.62343	0.62561	0.63403
c27	A7-OL1	0.01	0.84579	0.62111	0.63089	0.63842
c28	A7-OL1	0.05	0.84593	0.62147	0.62755	0.63452
c29	A7-OL1	0.1	0.84583	0.62105	0.58583	0.59298
c30	A7-OL1	0.5	0.84409	0.62071	0.46837	0.47337

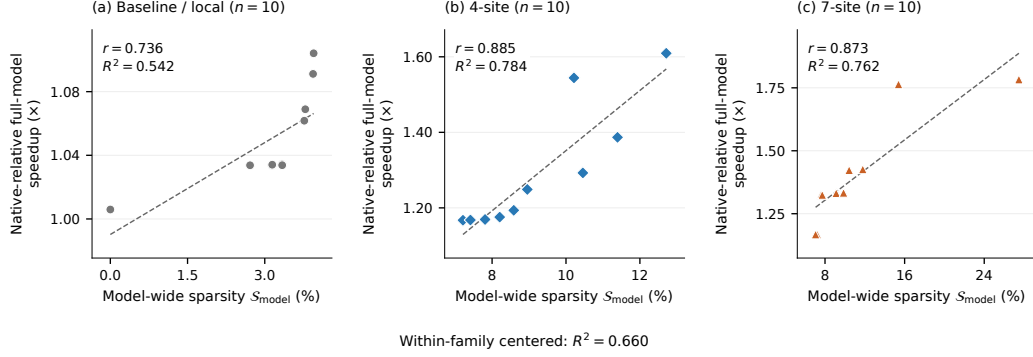


Figure 16: **Sparsity-speedup association within intervention families.** Each panel contains ten checkpoints and a descriptive OLS fit with an intercept, using each checkpoint’s own native-relative full K050 speedup. Axis ranges differ between panels to expose within-family variation. The centered statistic subtracts each family’s mean from both variables. These relationships persist within families but do not control for native cost, pressure, threshold or validation quality, and do not establish a causal effect of additional scalar zeros. Baseline/local comprises the dense reference, one-site ReLU, naive L1 and OL1.

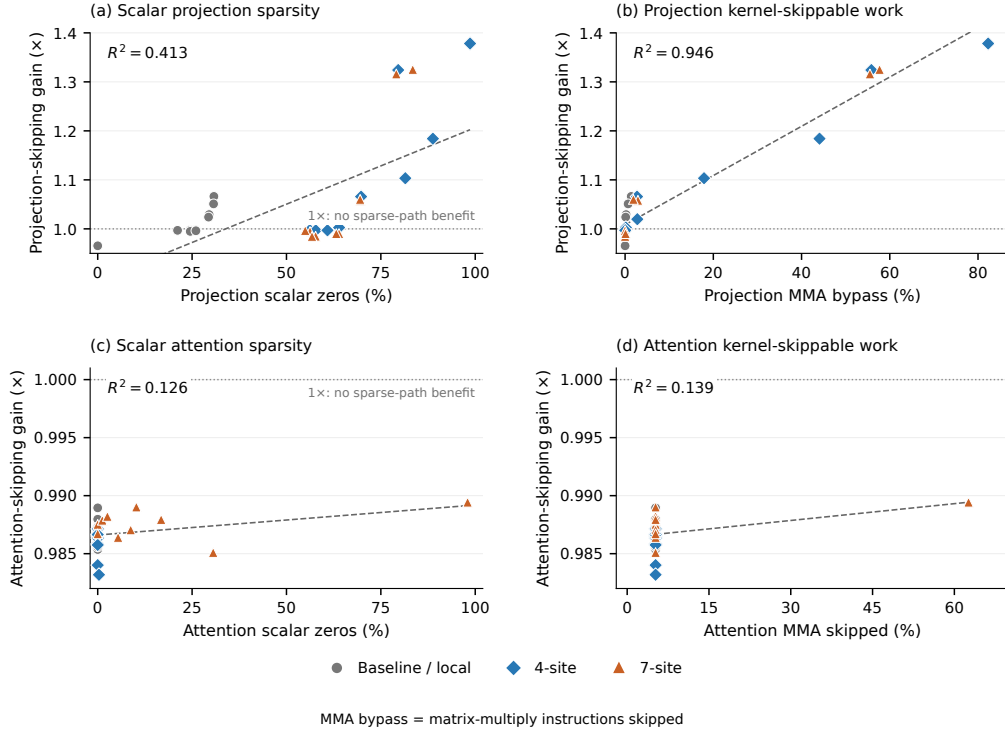


Figure 17: **Scalar zeros, skipped instructions and matched runtime gains.** All 30 checkpoints use the same family colors and shapes. Projection and attention gains are t_0/t_P and t_P/t_K , respectively; dashed lines are descriptive OLS fits with intercepts. Y-ranges match within rows. Scalar percentages use each operation group’s own product denominator, whereas instruction fractions retain tiled padding. Projection bypass includes scalar substitution, so bypass does not mean all corresponding arithmetic disappears. Attention counts include masked/padded work and have only two distinct predictor values. Counters cover 338 validation blocks and timings 64 matched inputs. Neither attention metric yields profitable skipping in this implementation.

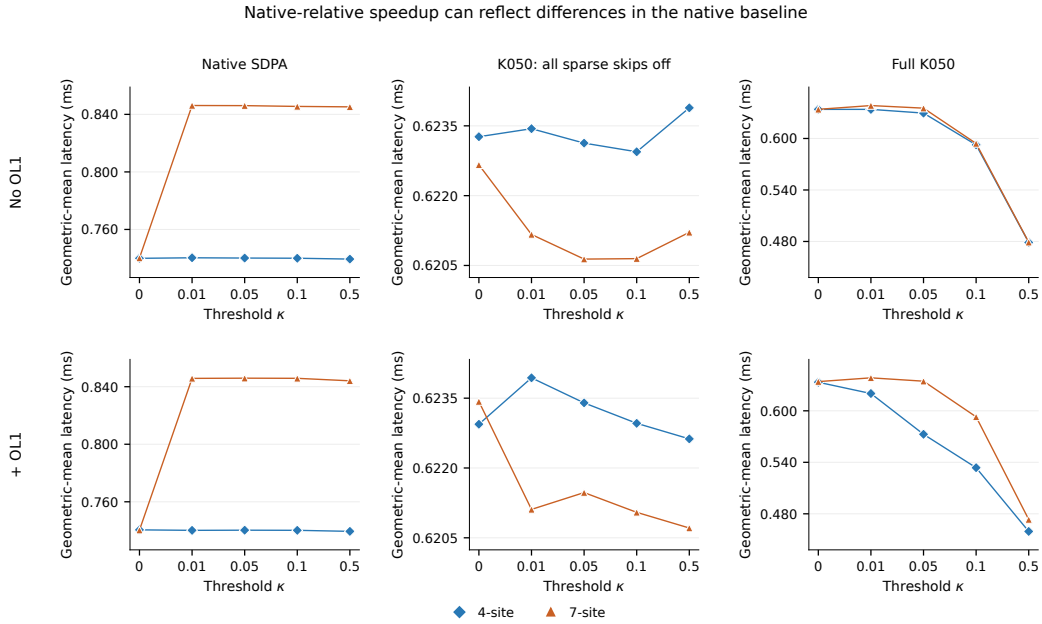


Figure 18: **Native reference costs and absolute optimized latency.** Four-/seven-site checkpoints are compared at the same threshold, without OL1 (top) and with OL1 (bottom). Columns show native SDPA, all-skips-off K050 and full K050 geometric-mean latencies. Y-ranges are shared within columns but differ across columns. The positive-threshold seven-site native overhead is much larger than the difference under the optimized implementation. Connecting lines order independently trained settings; they are not training trajectories or uncertainty estimates.

D.6 EVIDENCE AND DATA ACCOMPANYING THE DRAFT

The supplementary directory `supplementary-data/` contains all 54 training records with integer operation counts, all 540 thresholding evaluations and frontier memberships, seven signed histograms with zero and tail counts, figure coordinates, and the 30-checkpoint runtime reduction, with the kernel investigation’s per-checkpoint counters, raw-latency reductions, and stratified fits. The included README defines fields and units. Copy hashes are recorded in `SOURCES.json`; `protocol.json` gives data preparation, training settings and implementation identities. Paths refer to the accompanying repository; the supplementary directory is not a standalone checkpoint or kernel release.